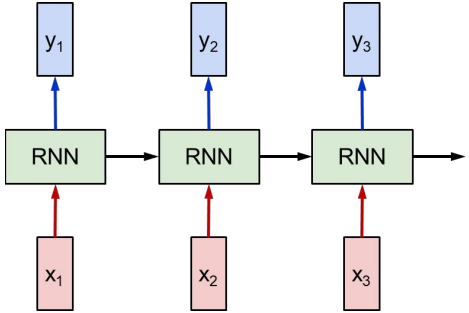
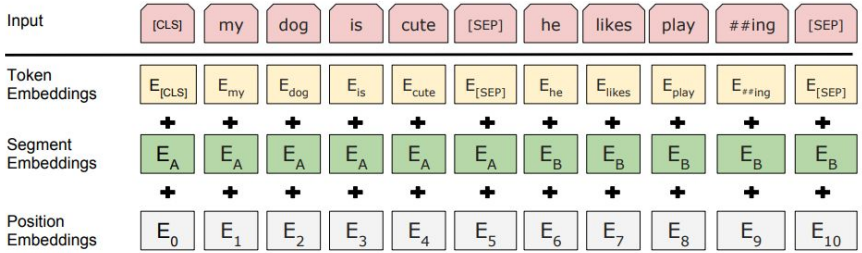
Sequence Modeling

Rui Cao
Assistant Professor
Information Systems Technology and Design

rui_cao@sutd.edu.sg
https://abril4416.github.io/

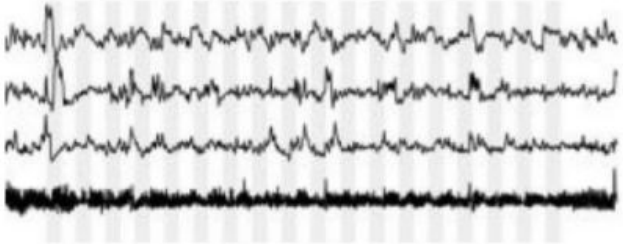




Examples of Sequences

"This morning I took the dog for a walk."

Sentences



Heart Rates

Audio Recordings

Sequence Modeling Tasks

- NLP: sentence completion
  - Word sequence

"This morning I took the dog for a walk."

given these words

predict what comes next?

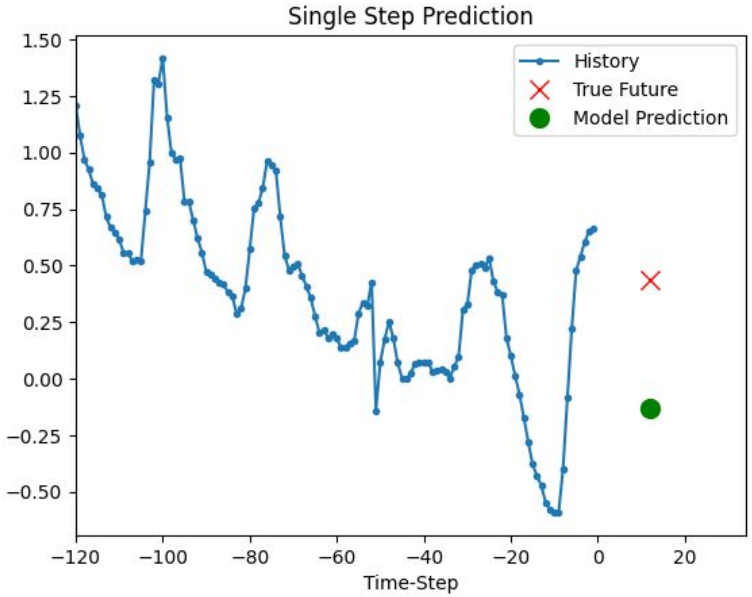
Sequence Modeling Tasks

- Multimodality: video captioning
  - Sequence of video frames, generated words

MPII-MD

Sequence Modeling Tasks

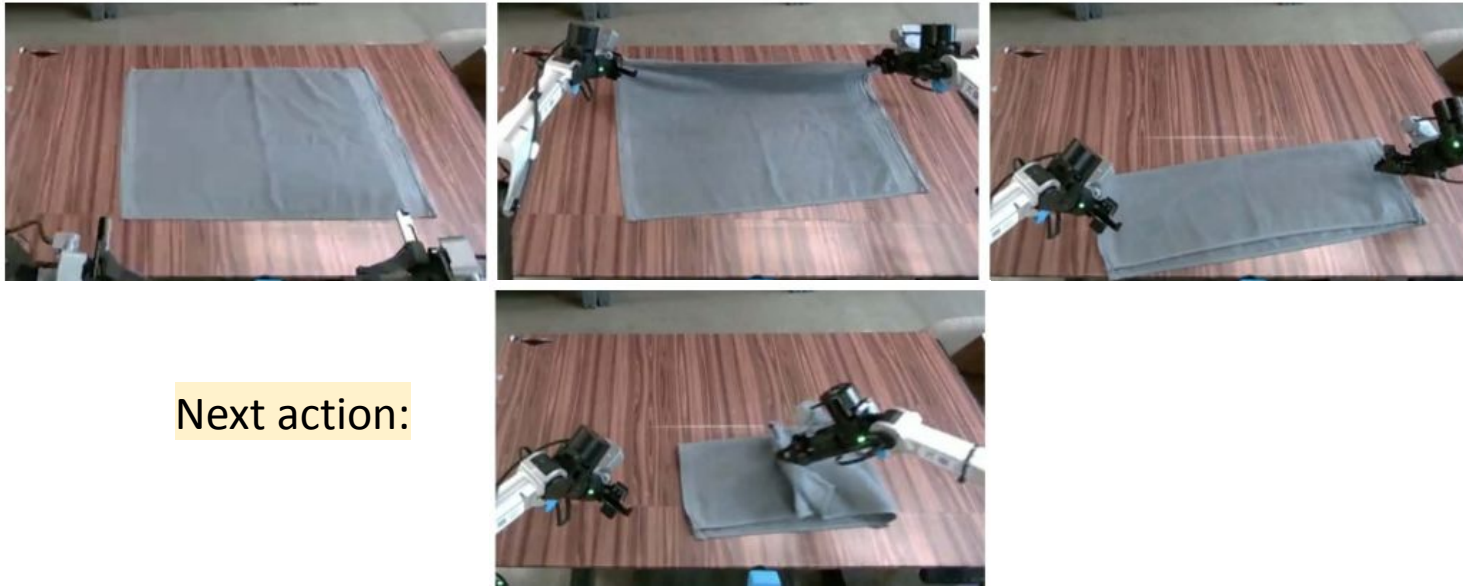
- Time series forecasting: weather forecasting
  - Sequence of measurements over time



Keras

Sequence Modeling Tasks

- Embodied AI: action prediction (e.g., towel folding)
  - Sequence of history observations, states and actions



\pi_0: towel folding

Question

- Apply what we have learnt?
  - Neural Networks (NN)?
  - Convolutional Neural Networks (CNN)?

image

Overview

- Neural Networks and Convolutional Neural Networks
  - How to apply
  - Limitations of NN/CNN for sequence modeling
- Recurrent Neural Networks
  - Architecture and training of RNN
  - Different types of RNN and applications
- Attention and Transformers
  - Attention mechanism and self-attention
  - Transformers

Overview

- Neural Networks and Convolutional Neural Networks
  - How to apply
  - Limitations of NN/CNN for sequence modeling
- Recurrent Neural Networks
  - Architecture and training of RNN
  - Different types of RNN and applications
- Attention and Transformers
  - Attention mechanism and self-attention
  - Transformers

Word Sequence in NLP

- A single modality: less complicated
- Widely studied and used: LLMs
  - The first type of sequence explored with Transformers
- As a representative type of inputs in sequence modeling
- **<u>Example used in this lecture</u>**

"This morning I took the dog for a walk."

given these words

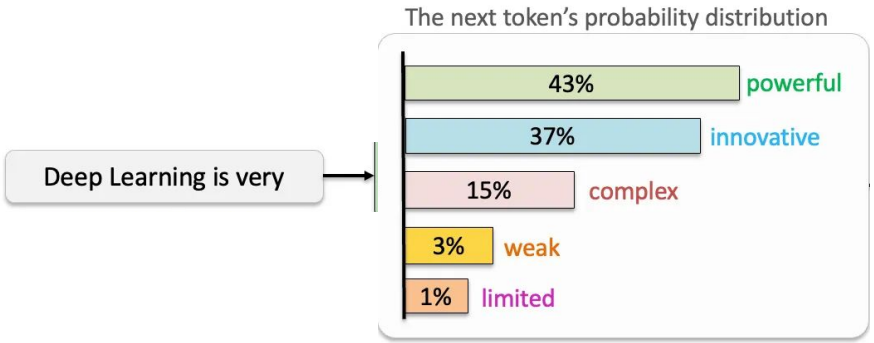
predict what comes next?

Sequence Modeling in NLP

- Language modeling
  - Input: a sequence of tokens (e.g., words, subwords)
  - Goal: learns the probability of the sequence
  - Common: next token prediction

Sequence Modeling in NLP

- Language modeling
  - Next token prediction
    - Compute sequence probabilities for each candidate word
    - Select the token to maximize the probability of the sequence

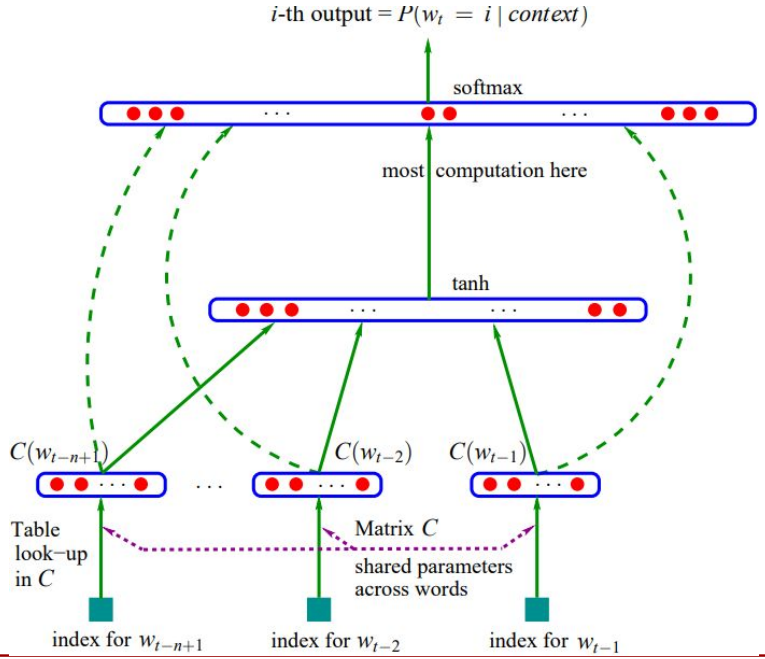


image

ANY QUESTION

NN for Language Modeling

- Take a fixed-length context window of previous words
  - Predict the next word

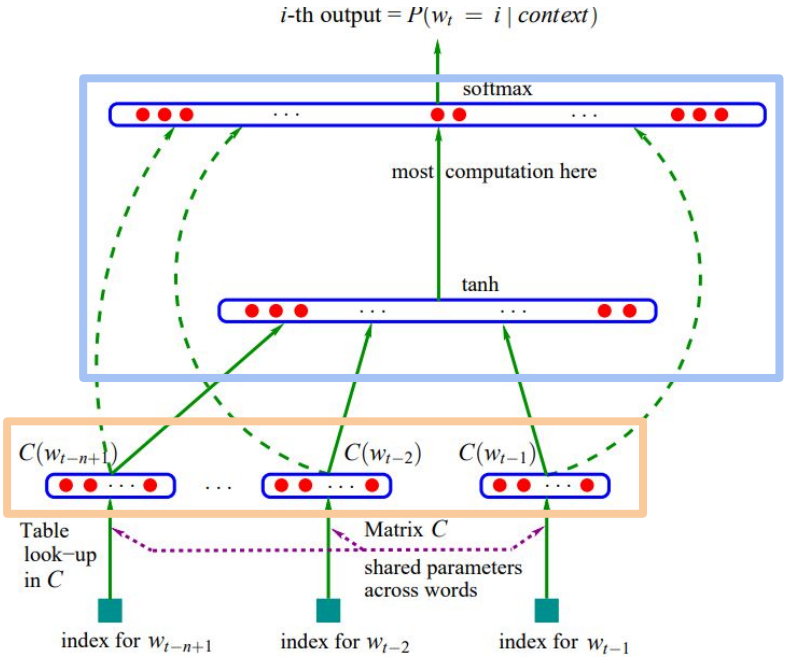


$$P(w_t \mid w_{t-1}, w_{t-2}, \ldots, w_{t-n+1})$$

2013 JMLR: A Neural Probabilistic Language Model

NN for Language Modeling

- Assume the window size is 3:
  - Concatenate word embeddings
  - Feed into an MLP to produce a probability distribution over vocabulary



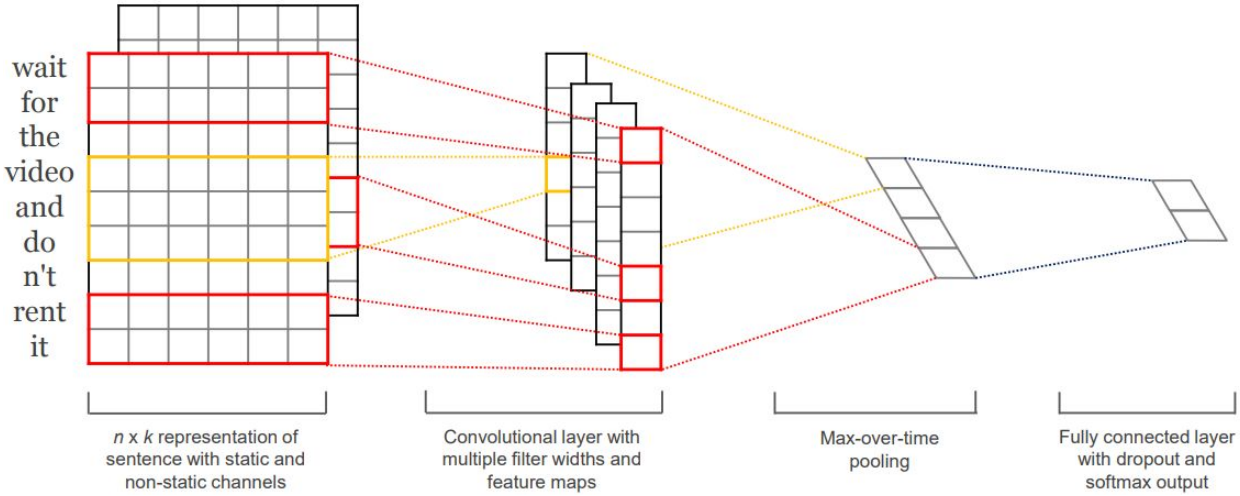
2013 JMLR: A Neural Probabilistic Language Model

NN for Language Modeling

- Motivation of the method
  - **Similarity** between words and contexts
  - Similar contexts may share similar hidden states
  - Similar probability distribution of the next token

CNN for Language Modeling

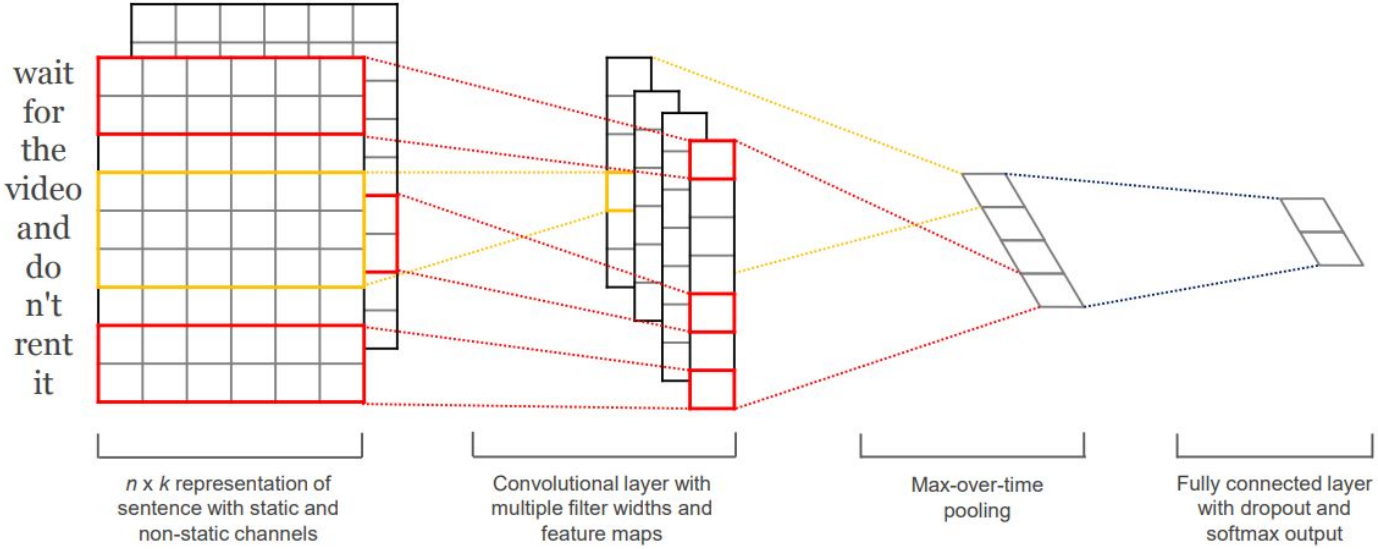
- Several filters with **varying window sizes**
  - More flexible than NN: contexts within different window sizes
  - Applied over word embeddings



2014 EMNLP Convolutional Neural Networks for Sentence Classification

CNN for Language Modeling

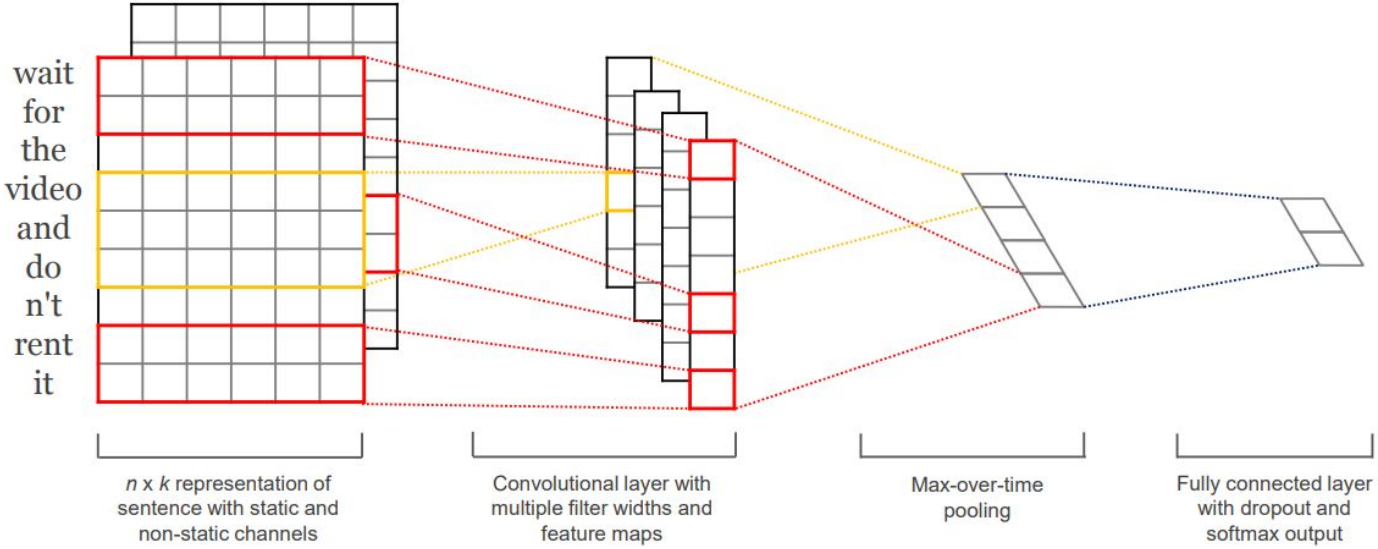
- Can use multiple channels: 2 here
  - One static (e.g., GloVe embeddings) one trainable



2014 EMNLP Convolutional Neural Networks for Sentence Classification

CNN for Language Modeling

- Aggregation over filters: max pooling
- Feed to a fully connected/linear layer for probability prediction



2014 EMNLP Convolutional Neural Networks for Sentence Classification

Comparison between NN and CNN

- Context window:
  - NN: one window size; CNN: multiple sizes (filter sizes)
- Parameter sharing:
  - NN: fully connected
  - CNN: locally connected
  - Specifically, with a context window/filter size of 3
    - NN: $W \in \mathbb{R}^{h \times 3d}$, h is the hidden dimension
    - CNN: $W \in \mathbb{R}^{k \times d}$, k=3 here

Limitation of NN/CNN

- **Long distance dependencies**
  - NN: Short and fixed context window; explosion of parameters with larger context windows (h*Kd)
  - CNN: longer dependencies require deeper networks

The **book** that the professor from Oxford recommended yesterday was **fascinating**

- **Lack of positional information**: no explicit modeling
  - E.g., The dog bit the man. vs. The man bit the dog

Overview

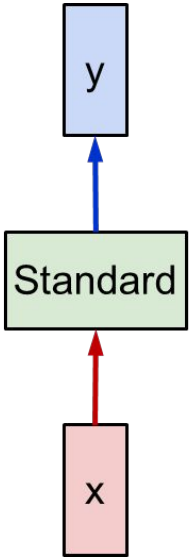
- Neural Networks and Convolutional Neural Networks
  - How to apply
  - Limitations of NN/CNN for sequence modeling
- Recurrent Neural Networks
  - Architecture and training of RNN
  - Different types of RNN and applications
- Attention and Transformers
  - Attention mechanism and self-attention
  - Transformers

Motivations behind RNN

- Inputs and output may not be of a fixed length
  - Input sentences can vary in length
- Want to model temporal aspect (sequence order) as context
  - Positional relationships between input tokens need to be captured
- Hard to determine appropriate window size for context
  - Dependencies may span varying context lengths
- Want to share parameters across sequence
  - The model should avoid parameter explosion as input length increases

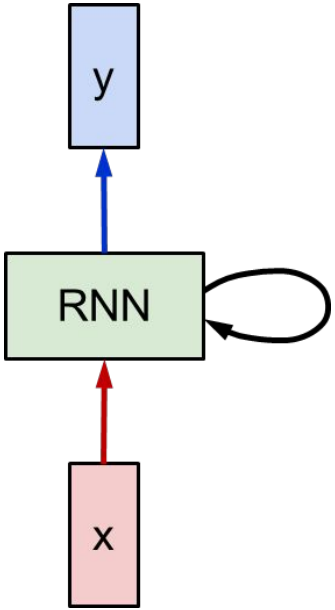
Recurrent Neural Networks: RNN

- Standard NN generate a single output



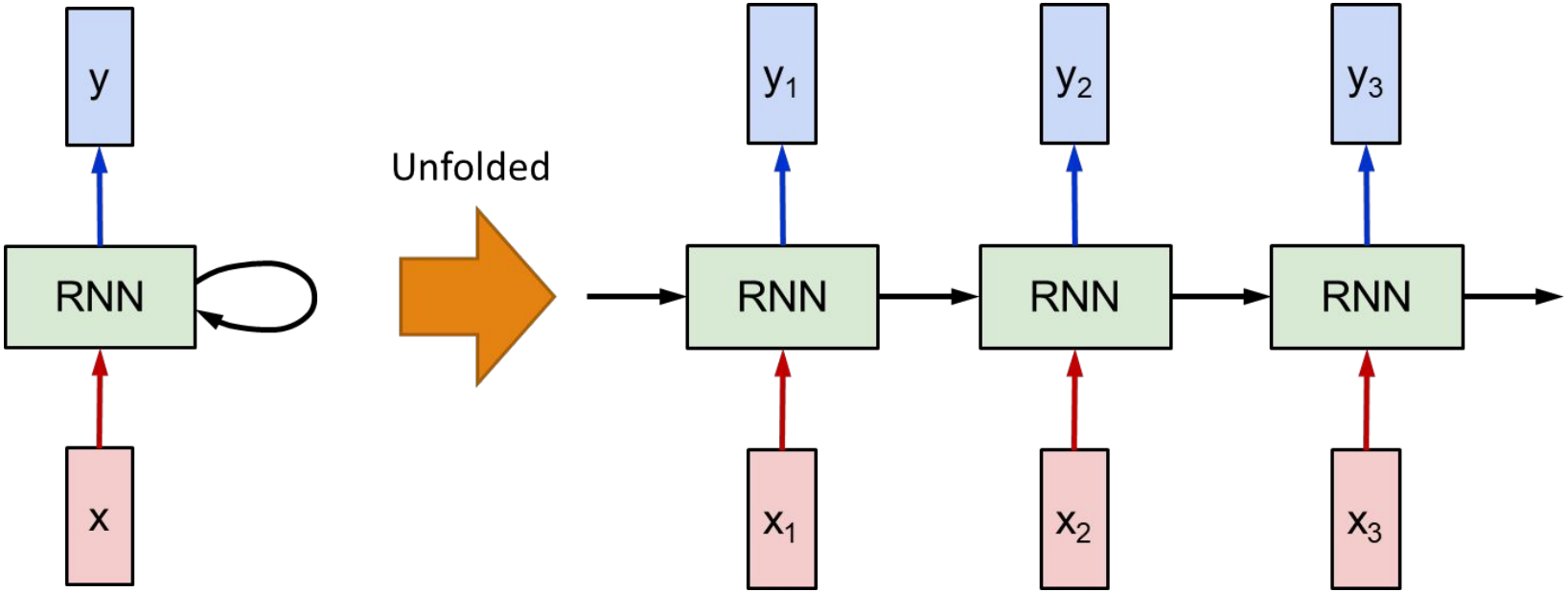
Recurrent Neural Networks: RNN

- RNN: aims to process a sequence of inputs to generate a sequence of outputs at different time steps



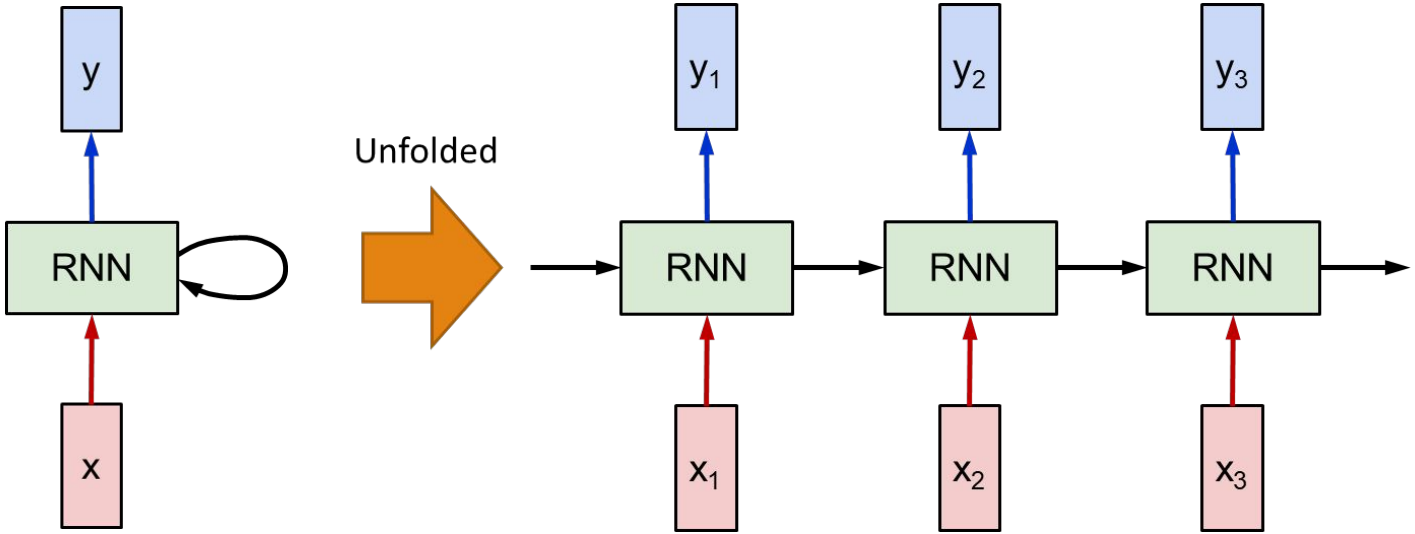
Recurrent Neural Networks: RNN

- RNN: aims to process a sequence of inputs to generate a sequence of outputs at different time steps



Recurrent Neural Networks: RNN

- Each step looks like a copy of the same model
- All copies share the same parameters
- Information flows from earlier steps to later ones



RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
    - <u>Hidden state: the model's memory of everything it has seen so far</u>

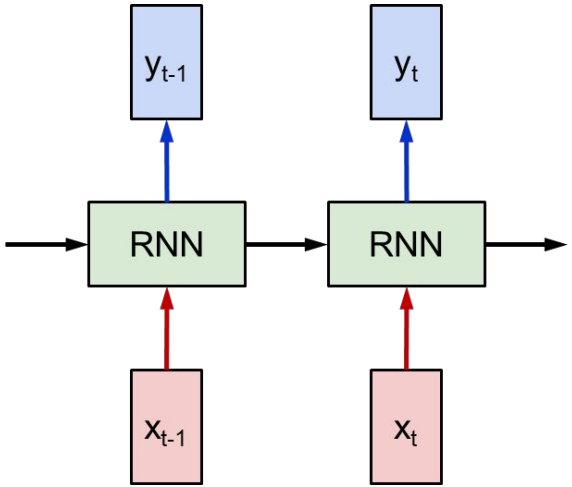
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

old state

input vector at some time step

some function with parameters W



RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
    - Previous hidden state, which carries information from earlier time steps

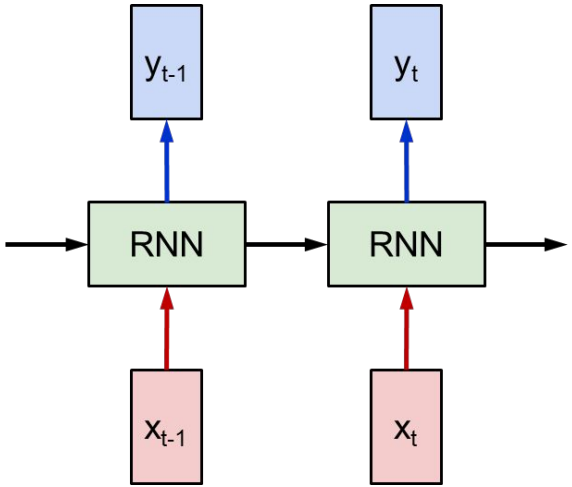
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

old state

input vector at some time step

some function with parameters W



RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
    - Current input, which provides new information at this step

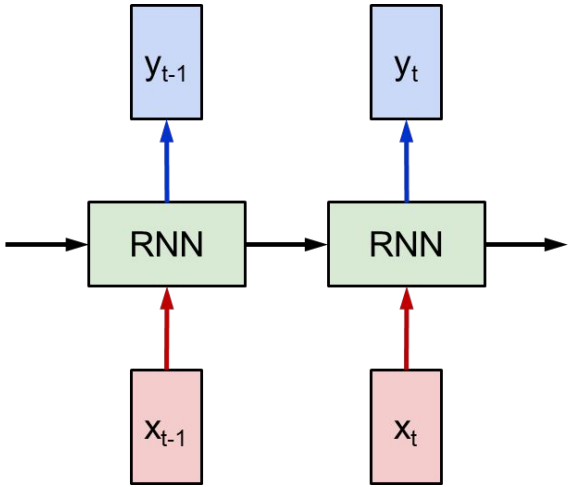
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

old state

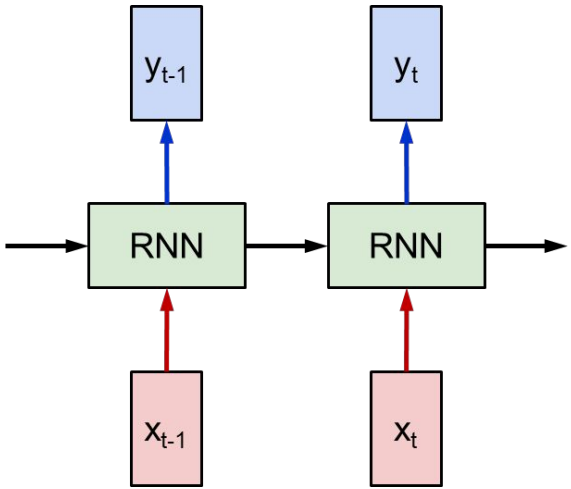
input vector at some time step

some function with parameters W



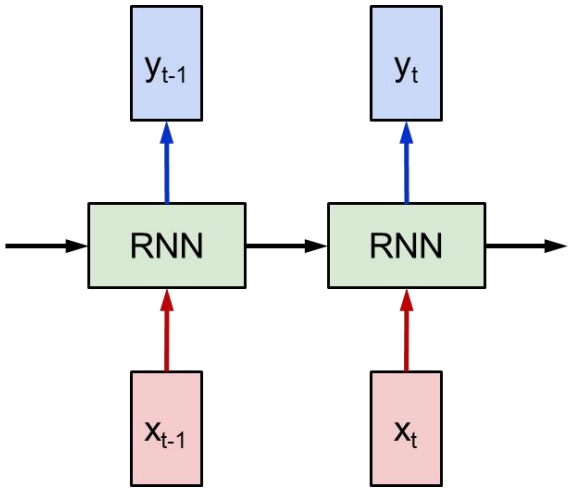
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - $h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$



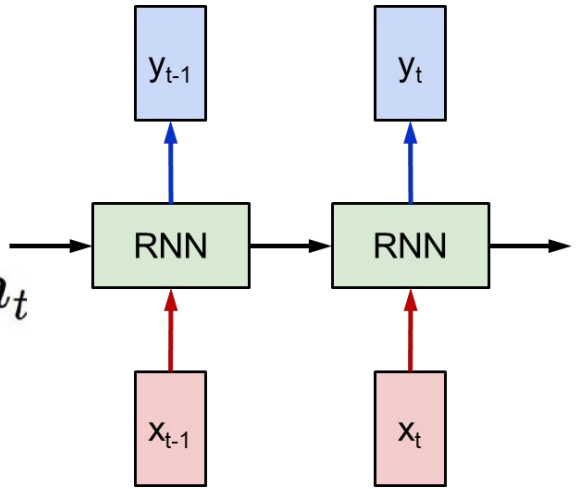
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - **Parameter sharing**: The same function f_W and parameters W are shared across time-steps



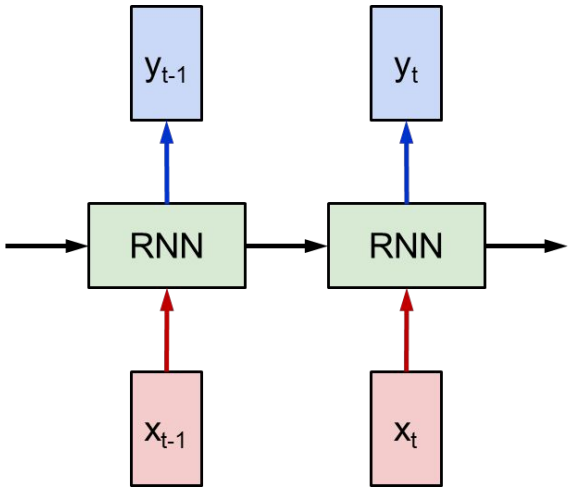
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - Task-specific output by applying FC/MLP over the current hidden state: $y_t = W_{hy} h_t$



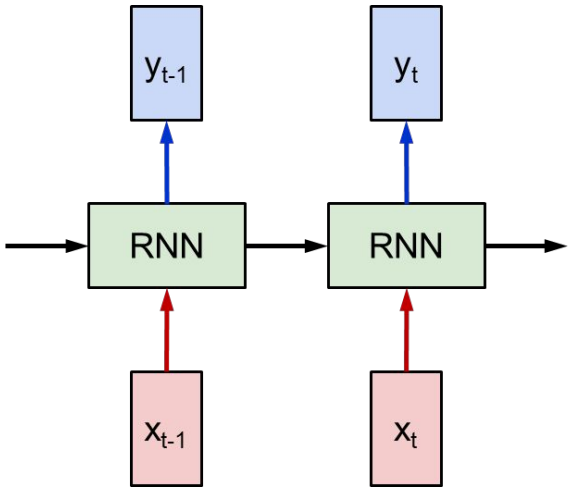
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - Task-specific output by applying FC over the current hidden state: $y_t = W_{hy} h_t$
    - Parameter sharing as well



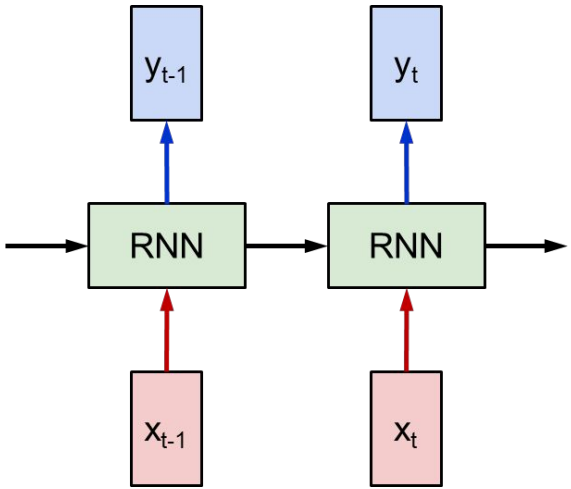
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - Task-specific output by applying FC over the current hidden state: $y_t = W_{hy} h_t$
    - Parameter sharing as well
    - Hidden state: a summary of information
    - Map to a task-specific prediction



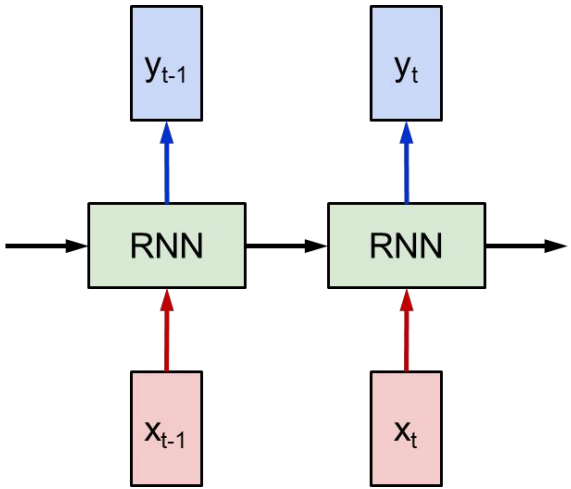
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - Task-specific output by applying FC over the current hidden state: $y_t = W_{hy} h_t$
    - Map to a task-specific prediction
      - E.g., next word prediction, sentiment analysis



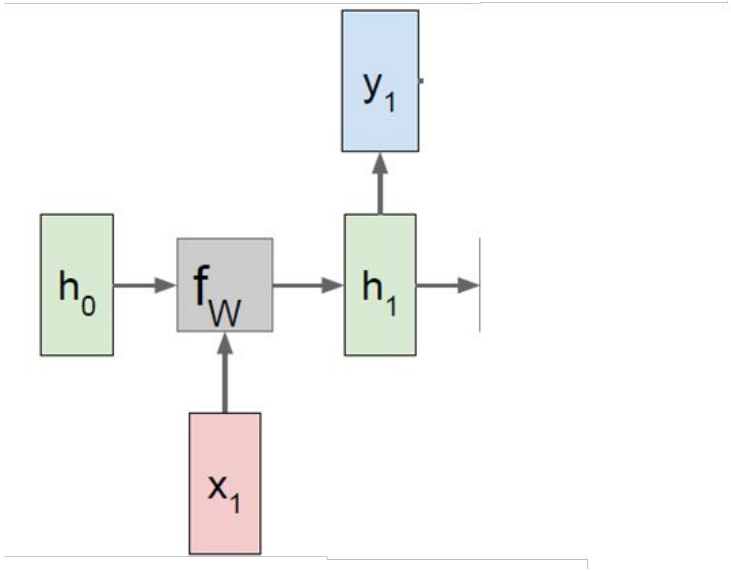
RNN Formula

- At time step t:
  - Generate a new hidden state given <u>previous hidden state</u> and <u>current input</u>
  - $h_t = f_W(h_{t-1}, x_t)$
  - Task-specific output $y_t = W_{hy} h_t$
    - Is a function of hidden state
      - History+current
    - Convert to task-specific prediction



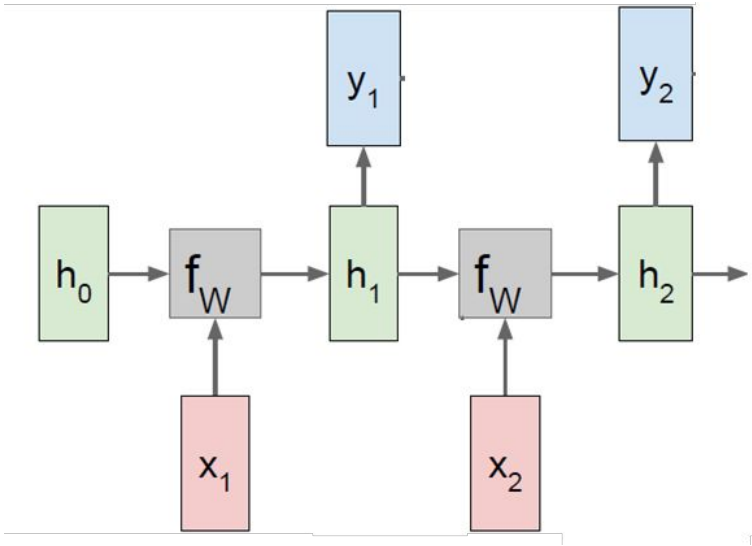
RNN Example: General

- Start from time-step 1
  - Output y_1



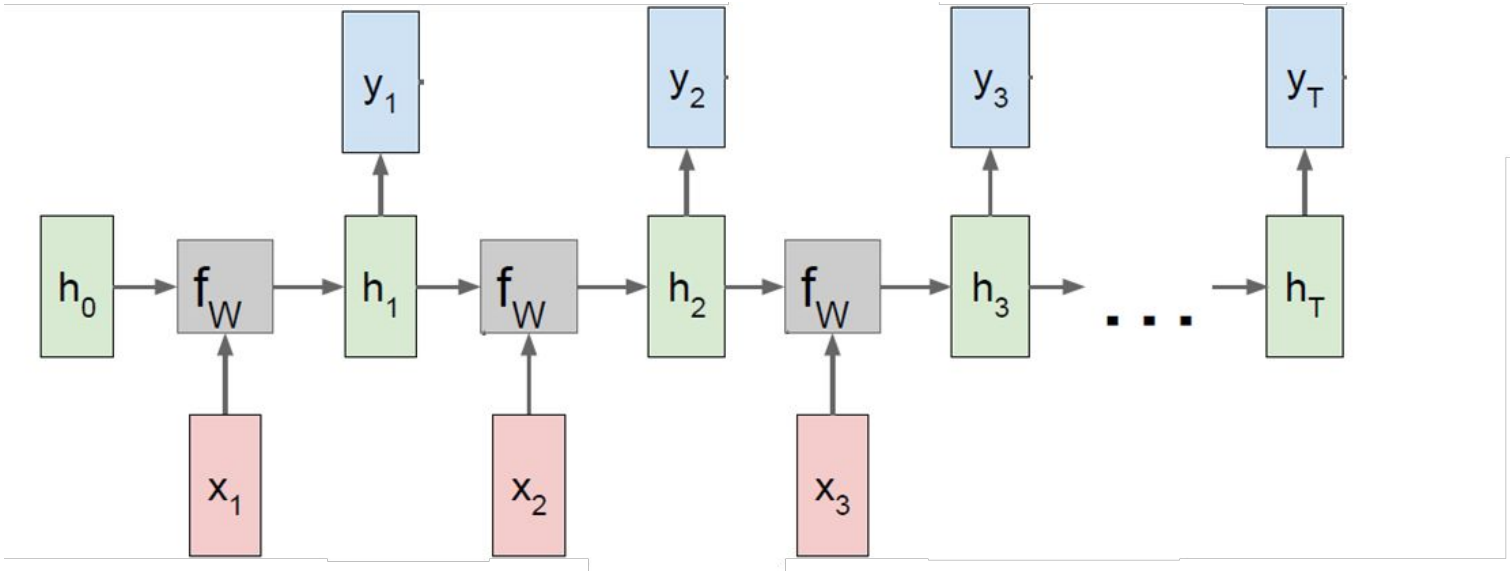
RNN Example: General

- Start from time-step 1, then time-step 2
  - Output y_1 y_2



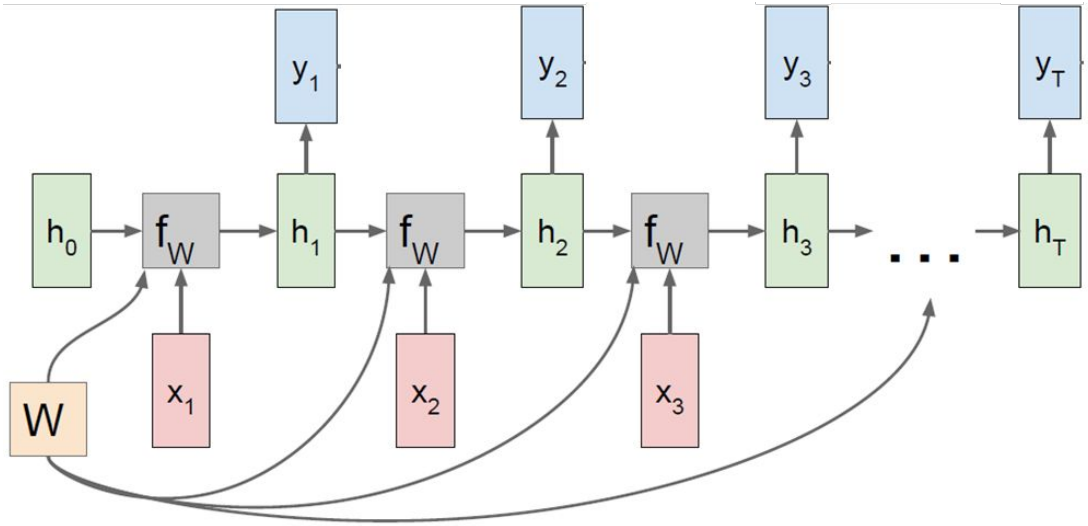
RNN Example: General

- Start from time-step 1, then time-step 2, until time-step T
  - Output $y_1\ y_2 \ldots y_T$



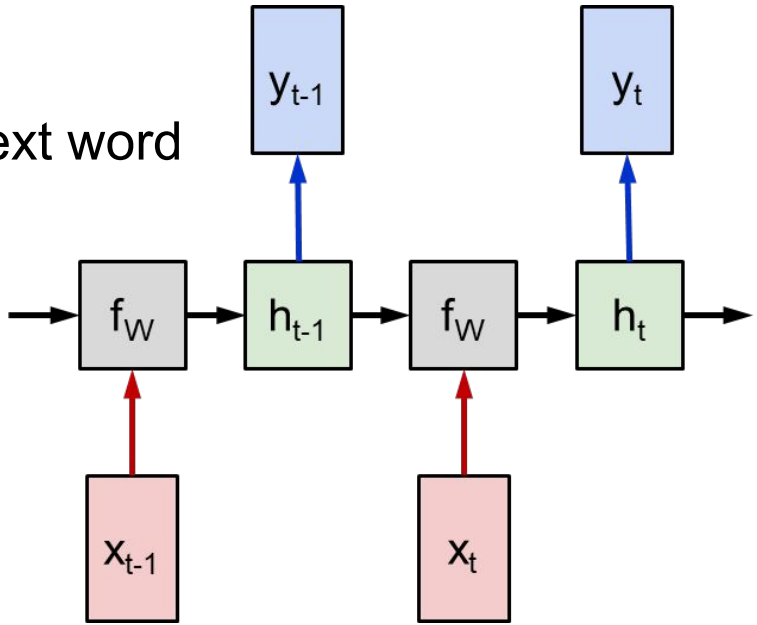
RNN Example: General

- Same weights W used throughout all time-steps
  - Hidden state computation; output layer



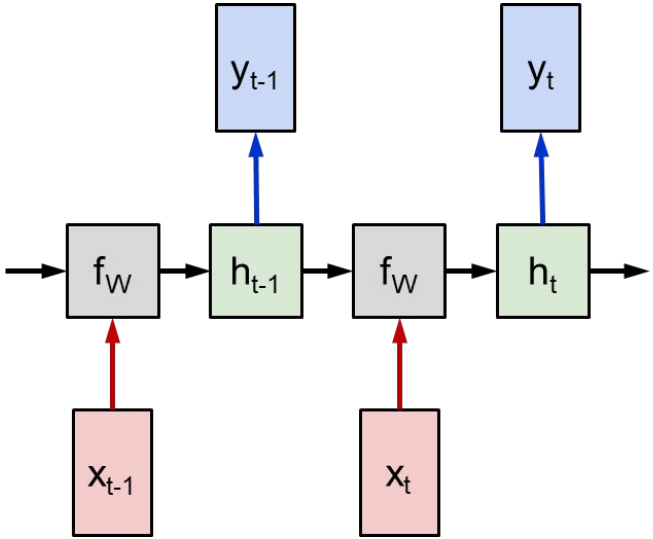
RNN Example

- Next word prediction
  - Receive t-2 previous tokens
  - Given the t-1^{-th} word, predict the next word
  - Probability of the next token y_{t-1}



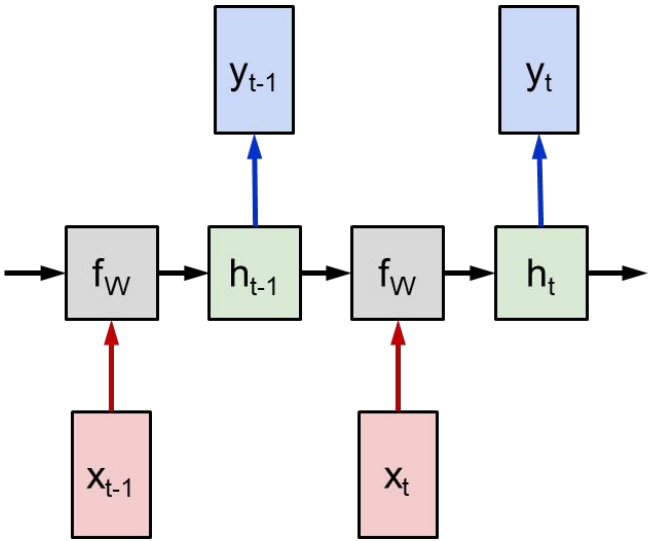
RNN Example

- Next word prediction, ***who is the***, t=4
  - Receive 2 previous tokens: ***who is***
  - Given the 3-rd word, ***the***
  - Predict the 4-th word



RNN Example

- Next word prediction t=4
  - Receive t-2 previous tokens: ***who is***
  - Given the t-1^{-th} word, ***the***
  - Probability of the next token y_{t-1}



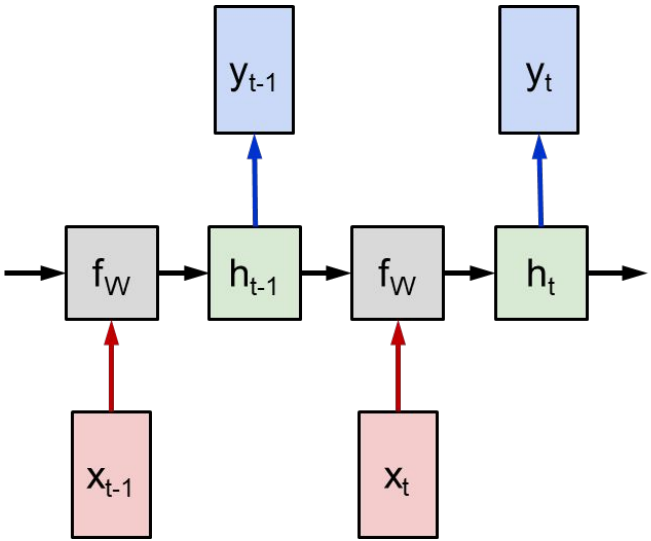
RNN Example

- Next word prediction t=4
  - Receive t-2 previous tokens: ***who is***
  - Given the t-1^{-th} word, ***the***
  - Probability of the next token y_{t-1}
    - $y_{t-1} = softmax(W_{hy} h_{t-1})$

$$h_1 = f(x_{\text{who}}, h_0)$$

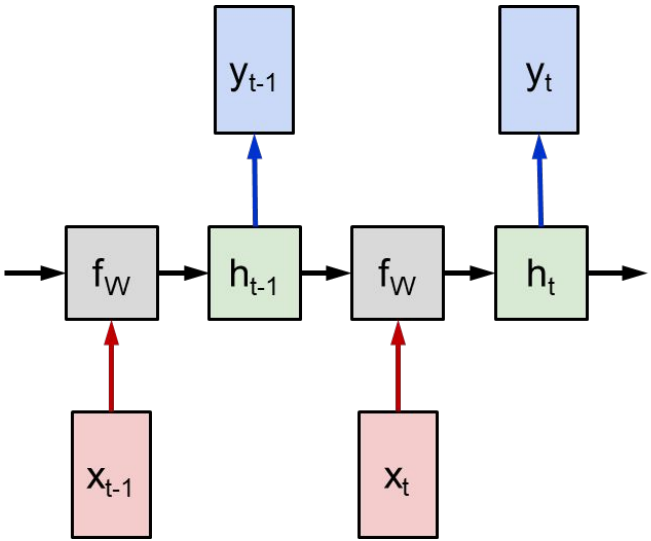
$$h_2 = f(x_{\text{is}}, h_1)$$

$$h_3 = f(x_{\text{the}}, h_2)$$



RNN Example

- Next word prediction t=4
  - Receive t-2 previous tokens: ***who is***
  - Given the t-1^{-th} word, ***the***
  - Probability of the next token y_3
    - $y_3 = softmax(W_{hy} h_3)$
    - $P(w_4 \mid \text{who is the}) = y_3$



ANY QUESTION

RNN Training

- Total loss comes from multiple time steps in a sequence
  - Each time step contributes its own loss
  - Combined to form the overall training objective

RNN Training

- Do not treat each time step separately: gradients are accumulated
  - $\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$, L_t denotes loss at the t-th time step
    - Gradient of a parameter is the sum of how that parameter affects the loss at each time step
    - Parameter sharing: every occurrence of the parameter contributes to the final gradien

RNN Training

- Do not treat each time step separately: gradients are accumulated
  - $\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$, L_t denotes loss at the t-th time step
    - Gradient of a parameter is the sum of how that parameter affects the loss at each time step
    - Parameter sharing: every occurrence of the parameter contributes to the final gradien
    - Each step ⇒ error ⇒ signal for parameter update ⇒ final training signal: combination fo all signals

RNN Training

- Gradients are accumulated: loss accumulated
  - $\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$, L_t denotes loss at the t-th time step
  - Hidden state and output:
    - $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
    - Previous hidden state and current input ⇒ new hidden state ⇒ map to output

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Output layer:
  - Error term definition: $\delta_t^y = \frac{\partial L_t}{\partial y_t}$
  - How much the output at time affects the loss
  - Starting point for backpropagation in chain rule

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Output layer:
  - Error term definition: $\delta_t^y = \frac{\partial L_t}{\partial y_t}$
  - Gradient : sum over time (each time step contributes a term)
    - Weights: $\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} \delta_t^y h_t^T$ (error * input, similar to standard NN)
    - Bias: $\frac{\partial L}{\partial b_y} = \sum_{t=1}^{T} \delta_t^y$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Output layer:
  - Error term definition: $\delta_t^y = \frac{\partial L_t}{\partial y_t}$
  - Gradient : sum over time (each time step contributes a term)
    - Weights: $\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} \delta_t^y h_t^T$ (error * input, similar to standard NN)
    - Bias: $\frac{\partial L}{\partial b_y} = \sum_{t=1}^{T} \delta_t^y$
    - Each time step, similar to NN; but accumulate over time
      - Parameter sharing
      - Backpropagation through time

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - Output at time t: contribute to current output

$$y_t = g(W_{hy}h_t + b_y)$$

$$\frac{\partial}{\partial h_t} = W_{hy}^T \frac{\partial}{\partial(W_{hy}h_t + b_y)}$$

$$\frac{\partial}{\partial h_t} = W_{hy}^T \left(\frac{\partial}{\partial y_t} \odot g'(W_{hy}h_t + b_y) \right)$$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - Output at time t: contribute to current output
  - **<u>Next hidden state</u>**: carry information to future time steps
    - Mistake now: propagate forward and influence future prediction

$$h_{t+1} = f(W_{xh}x_{t+1} + W_{hh}h_t + b_h)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial (W_{xh}x_{t+1} + W_{hh}h_t + b_h)} \right)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - Output at time t: contribute to current output
  - **<u>Next hidden state</u>**: carry information to future time steps
    - Additional source of error: later time steps

$$h_{t+1} = f(W_{xh}x_{t+1} + W_{hh}h_t + b_h)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial (W_{xh}x_{t+1} + W_{hh}h_t + b_h)} \right)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - Output at time t: contribute to current output
  - **<u>Next hidden state</u>**: carry information to future time steps
    - Gradient: the gradient from the next time step's hidden state and pass backward through the connection
      - Current to the next hidden

$$h_{t+1} = f(W_{xh}x_{t+1} + W_{hh}h_t + b_h)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial (W_{xh}x_{t+1} + W_{hh}h_t + b_h)} \right) \qquad \frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - Output at time t: contribute to current output
  - **<u>Next hidden state</u>**: carry information to future time steps
    - Gradient: the gradient from the next time step's hidden state and pass backward through the connection
      - Scaled by the weights, sensitivity to inputs (activation)

$$h_{t+1} = f(W_{xh}x_{t+1} + W_{hh}h_t + b_h)$$

$$\frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial (W_{xh}x_{t+1} + W_{hh}h_t + b_h)} \right) \qquad \frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

RNN Training

- Denotations: $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$ $y_t = g(W_{hy}h_t + b_y)$
- Hidden state at time step t: errors accumulate from two sources:
  - **<u>Output at time t</u>**: contribute to current output
  - **<u>Next hidden state</u>**: carry information to future time steps

$$\frac{\partial}{\partial h_t} = W_{hy}^T \left(\frac{\partial}{\partial y_t} \odot g'(W_{hy}h_t + b_y) \right) \quad \frac{\partial}{\partial h_t} = W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

$$\frac{\partial}{\partial h_t} = W_{hy}^T \left(\frac{\partial}{\partial y_t} \odot g'(W_{hy}h_t + b_y) \right) + W_{hh}^T \left(\frac{\partial}{\partial h_{t+1}} \odot f'(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \right)$$

Updating Rule

- Define $z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$
  - Update weights: accumulate contributions from all time steps
    - the gradient at that step and the value of the previous hidden state

$$\frac{\partial}{\partial W_{hh}} = \sum_t \left(\frac{\partial}{\partial z_t}\right) h_{t-1}^T$$

  - The gradient with respect to the hidden state and scaling it by how sensitive the activation function is at that point.

$$\frac{\partial}{\partial z_t} = \frac{\partial}{\partial h_t} \odot f'(z_t)$$

Vanishing Gradient in RNN

- Define $z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$ $\quad h_t = \boxed{f}(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$

$$\frac{\partial}{\partial z_t} = \frac{\partial}{\partial h_t} \odot \boxed{f'(z_t)}$$

$$\frac{\partial L}{\partial z_t} = \left(\frac{\partial L_t}{\partial h_t} + W_{hh}^{\top}\frac{\partial L}{\partial z_{t+1}}\right) \odot f'(z_t)$$

$$\frac{\partial}{\partial W_{hh}} = \sum_t \boxed{\left(\frac{\partial}{\partial z_t}\right)} h_{t-1}^T$$

Gradient is scaled by how sensitive the activation function is: Tanh function

Derivative of tanh is (0,1]; keep shrinking

Long sequence, earlier steps=> gradient closer to 0

Vanishing Gradient in RNN

- Define $z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$ $\quad h_t = \boxed{f}(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$

$$\frac{\partial}{\partial z_t} = \frac{\partial}{\partial h_t} \odot \boxed{f'(z_t)}$$

$$\frac{\partial L}{\partial z_t} = \left(\frac{\partial L_t}{\partial h_t} + W_{hh}^{\top}\frac{\partial L}{\partial z_{t+1}}\right) \odot f'(z_t)$$

$$\frac{\partial}{\partial W_{hh}} = \sum_t \boxed{\left(\frac{\partial}{\partial z_t}\right)} h_{t-1}^T$$

Vanishing gradient: struggle with long-term dependencies
Earlier steps: weak training signals from later output

Solution: Gating Connection

- Gated Recurrent Unit (GRU)
  - Update gate (previous hidden) $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
  - Reset gate $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
  - Candidate hidden state $\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$
  - Final hidden state $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

Solution: Gating Connection

- Gated Recurrent Unit (GRU) $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
  - **<u>Adaptive memory update</u>**, depending on z_t
    - Near 0, keep old memory; near 1, overwrite memory

Solution: Gating Connection

- Gated Recurrent Unit (GRU)
  - Adaptive memory update: z_t $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
    - Near 0, keep old memory; near 1, overwrite memory
  - **<u>Controlled forgetting:</u>** r_t $\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
    - Near 0, ignore previous memory; near 1, use past memory

Solution: Long Short-Term Memory

- Explicit cell memory
- Three gates:
  - Input (i_t): how much input enters memory
  - Forget (f_t): how much memory to be kept
  - Output (o_t): determines which part of memory becomes hidden state

$$
\begin{aligned}
f_t &= \sigma_g\left(W_f x_t + U_f h_{t-1} + b_f\right) \\
i_t &= \sigma_g\left(W_i x_t + U_i h_{t-1} + b_i\right) \\
o_t &= \sigma_g\left(W_o x_t + U_o h_{t-1} + b_o\right) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right) \\
h_t &= o_t \circ \sigma_h\left(c_t\right)
\end{aligned}
$$

Solution: Long Short-Term Memory

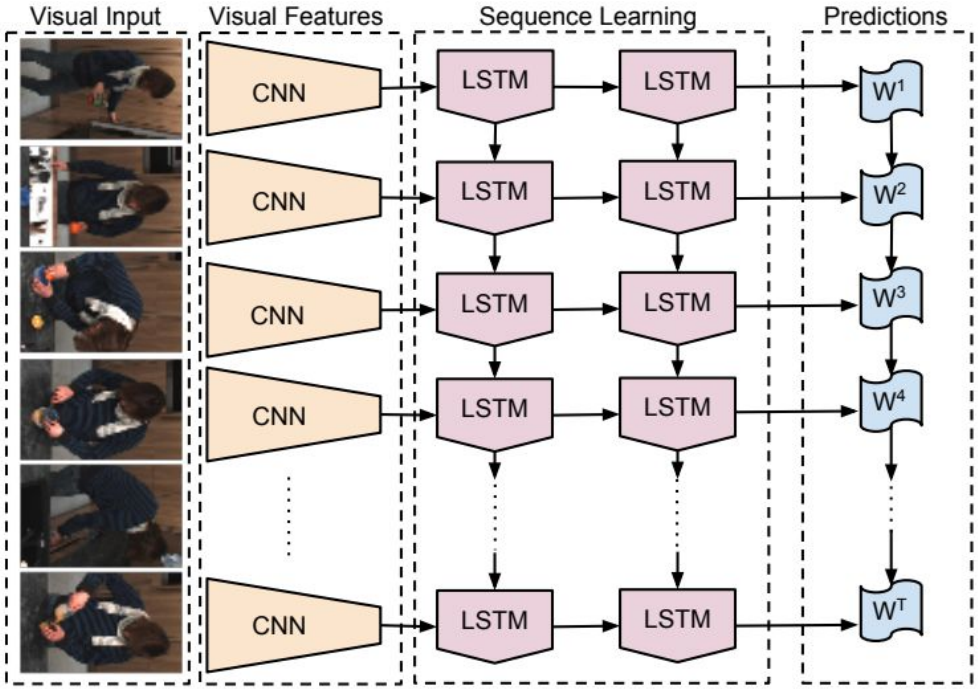
- Explicit cell memory
  - Combines old and new memory
  - What do I remember overall
  - Mitigate vanishing gradient: addictive path
  - Recurrent equations

$$
\begin{aligned}
f_t &= \sigma_g\left(W_f x_t + U_f h_{t-1} + b_f\right) \\
i_t &= \sigma_g\left(W_i x_t + U_i h_{t-1} + b_i\right) \\
o_t &= \sigma_g\left(W_o x_t + U_o h_{t-1} + b_o\right) \\
\boxed{c_t} &\boxed{= f_t \circ c_{t-1} + i_t \circ \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right)} \\
h_t &= o_t \circ \sigma_h\left(c_t\right)
\end{aligned}
$$

ANY QUESTION

Applications of RNN

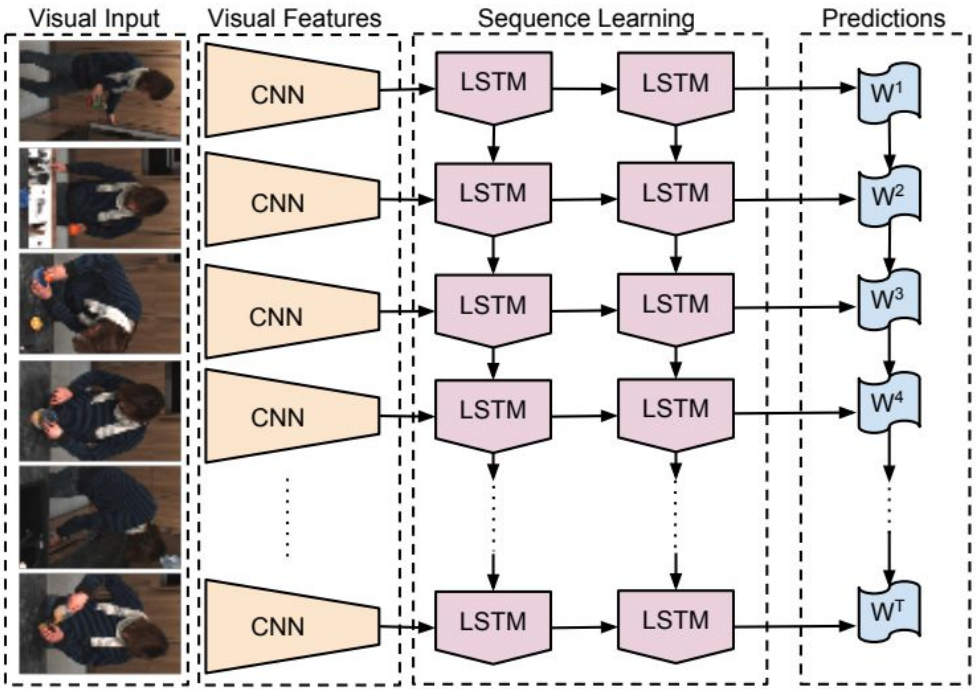
- Video captioning:
  - A sequence of frames



2015 CVPR Long-term Recurrent Convolutional Networks for Visual Recognition and Description

Applications of RNN

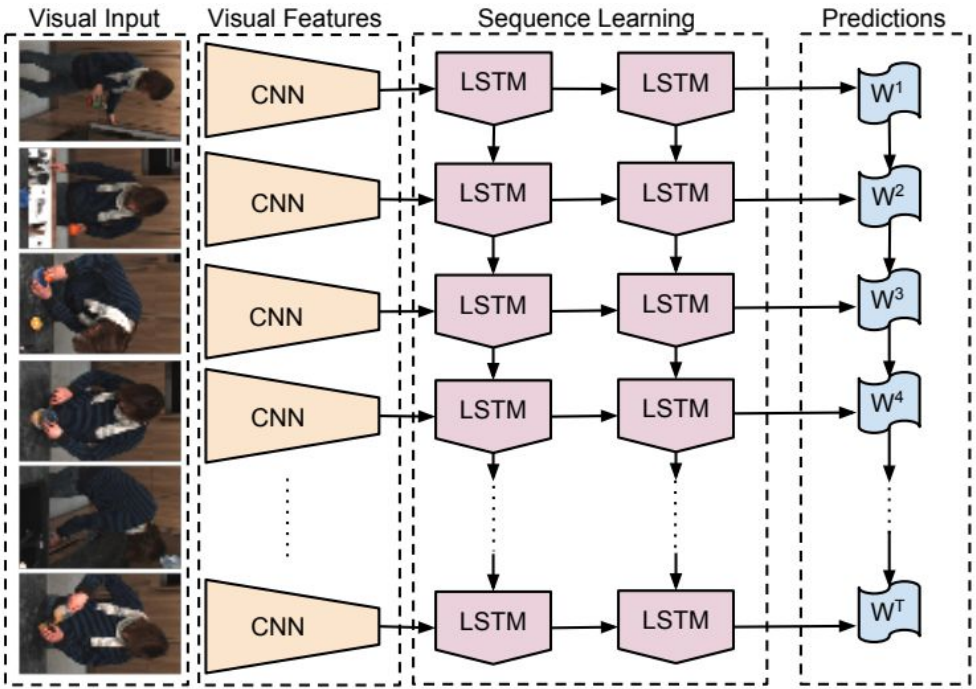
- Video captioning:
  - First LSTM: receive current image feature



2015 CVPR Long-term Recurrent Convolutional Networks for Visual Recognition and Description

Applications of RNN

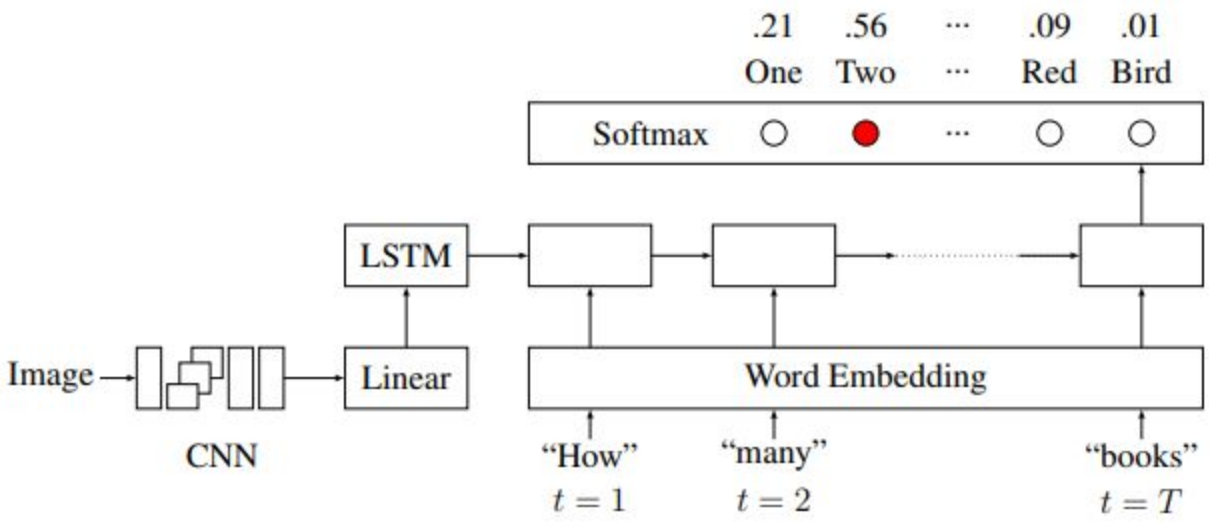
- Video captioning:
  - First LSTM: receive current image feature
  - Second LSTM: deeper model, increasing representational capacity
  - Higher level-events or semantic concepts



2015 CVPR Long-term Recurrent Convolutional Networks for Visual Recognition and Description

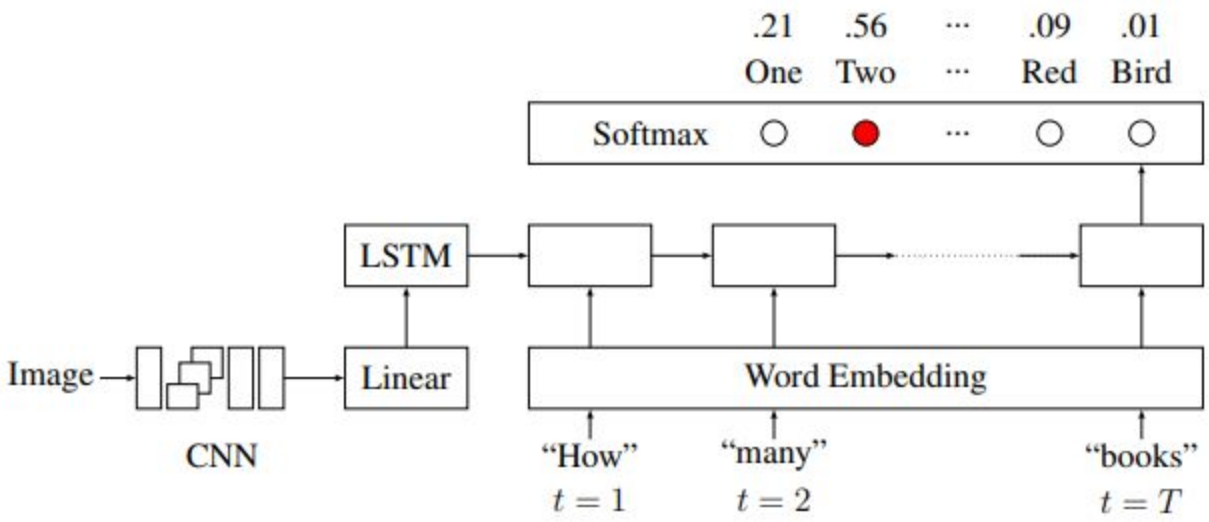
Applications of RNN

- Image question answering: image as the first token



Applications of RNN

- Final step: output probability distribution over answer vocabulary



Limitation:

- What is the limitation of previous image QA architecture?

What color is the ball?

Limitation: Uniform Attention

- Image captioning
  - Different generation step should attend to different regions

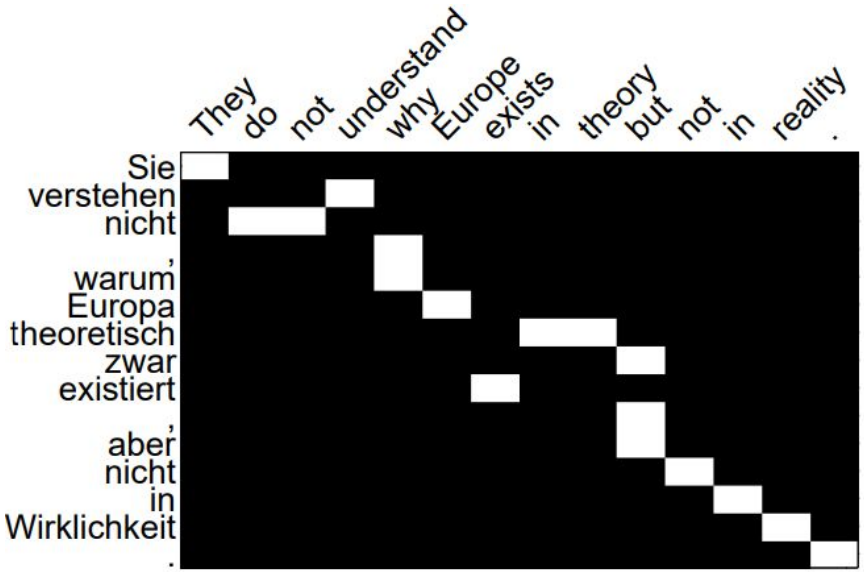
A dog is standing on a hardwood floor.

A stop sign is on a road with a mountain in the background.

2015 ICML Show, Attend and Tell: Neural Image Caption Generation with Visual Attention

Limitation: Uniform Attention

- Machine translation
  - Not all input words are equally important at each step



2015 EMNLP Effective Approaches to Attention-based Neural Machine Translation

Overview

- Neural Networks and Convolutional Neural Networks
  - How to apply
  - Limitations of NN/CNN for sequence modeling
- Recurrent Neural Networks
  - Architecture and training of RNN
  - Different types of RNN and applications
- Attention and Transformers
  - Attention mechanism and self-attention
  - Transformers

Attention Mechanism

- Goal: dynamically select and weight the most relevant parts of its input (or internal representations) when producing an output
  - Weights over a set of representations based on their relevance to a query
    - q is the query representation
    - k_i is the i-th representation
    - s is the relevance function

$$\alpha_i = \mathrm{softmax}(s(q, k_i))$$

Attention Mechanism

- Goal: dynamically select and weight the most relevant parts of its input (or internal representations) when producing an output
  - Weights over a set of representations based on their relevance to a query
    - q is the query representation (query vector)
    - k_i is the i-th representation (key vector)
    - s is the relevance function

Can you think of any methods for computing relevance?

$$\alpha_i = \mathrm{softmax}(s(q, k_i))$$

Examples: Attention

- Image question answering:
  - Query is the question representation
  - Attention weights computed over image regions

What color is the ball?

Examples: Attention

- Image question answering:
  - Query is the question representation
  - Attention weights computed over image regions

What color is the ball?

Examples: Attention

- Image captioning
  - Query vector: hidden representation from t-1
  - Attention weights computed over image regions

A dog is standing on a hardwood floor.

A stop sign is on a road with a mountain in the background.

Attention: Definition

- Query vector
  - What the model is currently focusing on
- Key vector
  - Candidates to compare against
- Attention score:
  - Relevance of each candidate key to query
- Value vector
  - Information to be aggregated

Attention Score Function

- Measures how relevant a query q is to a key k
- Dot-product $q^T k$
  - Fast and simple: but q and k should be of the same dimension

$$\alpha_i = \text{softmax}(s(q, k_i))$$

Attention Score Function

- Measures how relevant a query q is to a key k
- Dot-product
- Scaled dot-product: normalized by key dimension $q^T k / \sqrt{d_k}$
  - More stable for large models: used in Transformers
  - Cosine similarity $s(q,k) = \frac{q^T k}{\|q\| \|k\|}$

$$\alpha_i = \text{softmax}(s(q, k_i))$$

Attention Score Function

- Measures how relevant a query q is to a key k
- Dot-product
- Scaled dot-product
- MLP-based attention
  - q and k of different dimensions; more computationally intensive
    - [;] denotes concatenation: $s(q, k) = \mathrm{MLP}([q; k])$
    - v^T is the weight matrices: $v^T \tanh(W_q q + W_k k)$

$$\alpha_i = \mathrm{softmax}(s(q, k_i))$$

Attention Score Function

- Measures how relevant a query q is to a key k
- Dot-product
- Scaled dot-product
- MLP-based attention
- Bilinear attention $q^T W k$
  - Allow element-wise interaction ($q \in R^{n1}$, $k \in R^{n2}$)
  - W of high dimension (**<u>n1*n2*n3</u>**, n3 is the output dimension)

$$\alpha_i = \text{softmax}(s(q, k_i))$$

Attention Score Function

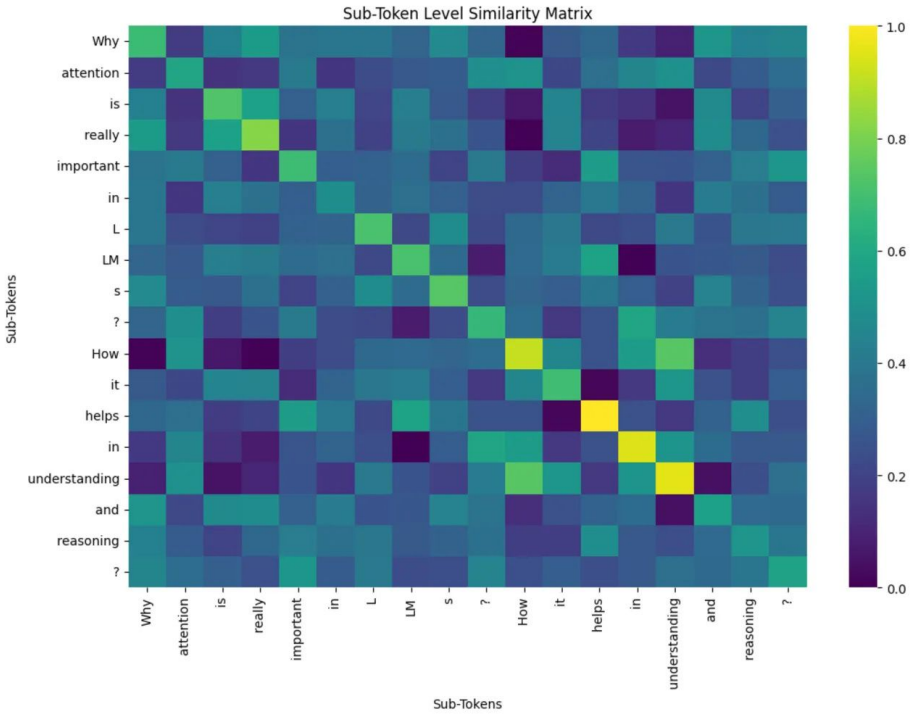
- Measures how relevant a query q is to a key k
- Dot-product
- Scaled dot-product
- MLP-based attention
- Bilinear attention $q^T W k$
  - Used in early stage attention
  - Tensor decomposition to reduce number of parameters

$$\alpha_i = \mathrm{softmax}(s(q, k_i))$$

ANY QUESTION

Self-Attention

- Every element in a sequence can interact with every other element in the same sequence
  - Relation between words



Self-Attention

- Queries, keys and values are derived from the same input sequence
  - Every element: three different representations
- Assume an input sequence $X = (x_1, x_2, \ldots, x_n)$
  - Linear transformation: $q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$
  - Three separate linear transformations: different roles in attention

Self-Attention

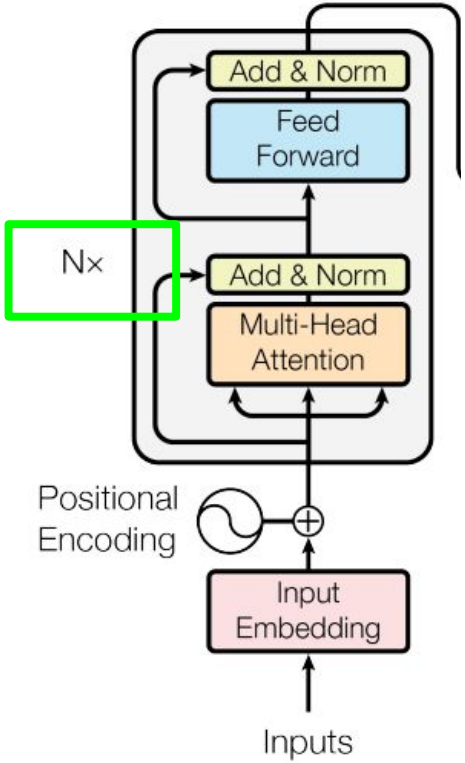
- Queries, keys and values are derived from the same input sequence
  - Every element: three different representations
- Assume an input sequence $X = (x_1, x_2, \ldots, x_n)$
  - Linear transformation: $q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$
  - Three separate linear transformations: different roles in attention
    - Query: attention requests
    - Keys: candidate elements to be compared with queries
    - Value: information to be aggregated (attention weights to be applied over)

Self-Attention

- Queries, keys and values are derived from the same input sequence
  - Every element: three different representations
- Assume an input sequence $X = (x_1, x_2, \ldots, x_n)$
  - Linear transformation: $q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$
  - Scaled dot-product: $\alpha_{ij} = \mathrm{softmax}\left(\frac{q_i^T k_j}{\sqrt{d_k}}\right)$
  - Aggregation: weighted sum of all values $h_i = \sum_{j=1}^{n} \alpha_{ij} v_j$

Transformer

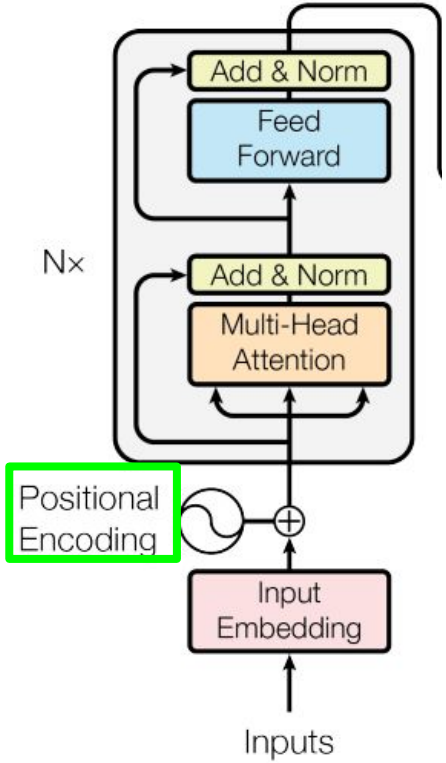
- A sequence to sequence model based entirely on attention
  - Stack of the same block N times



2017 NeurIPs Attention Is All You Need

Transformer

- A sequence to sequence model based entirely on attention
- Components:
  - Positional encoding: data order
  - Scaled dot-produce self-attention: contextual
  - Multi-head attention: relationships
  - Residual & Layer normalization
  - Feed forward networks (FFN)



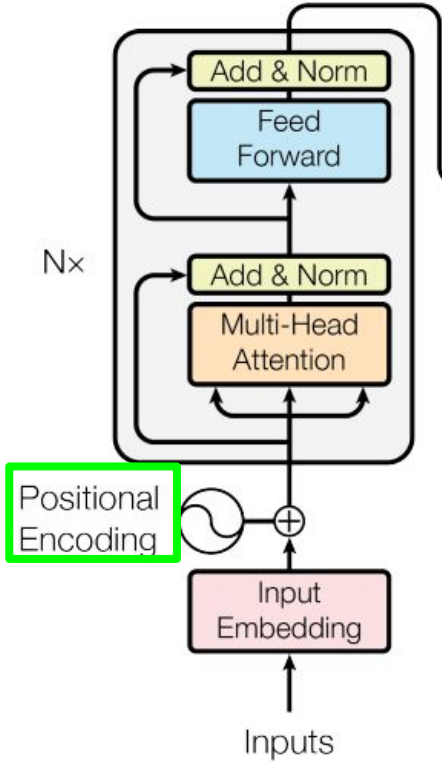
2017 NeurIPs Attention Is All You Need

Transformer

- Positional encoding
  - Identify the position of each token
  - <u>Sinusoidal encoding</u>: calculate each dimension with a sinusoidal function, sin/cos function

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

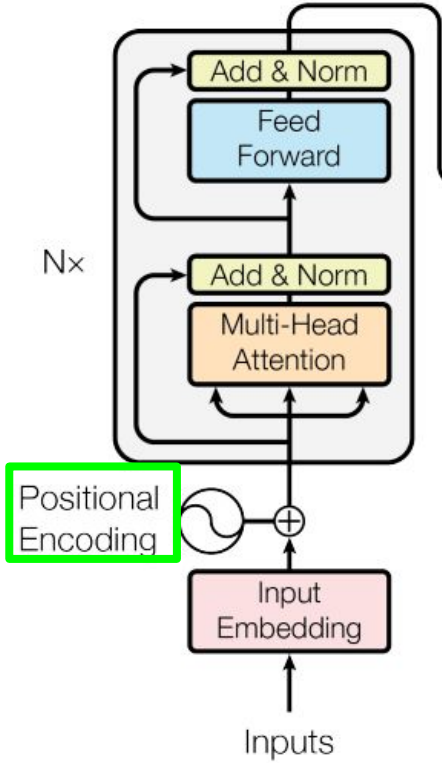
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



2017 NeurIPs Attention Is All You Need

Transformer

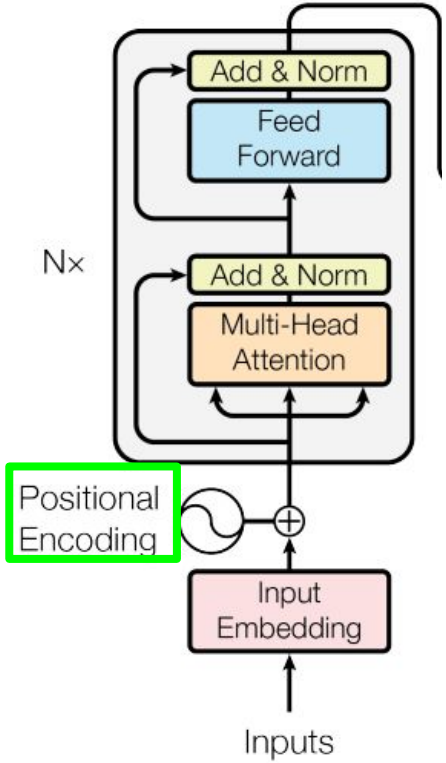
- Positional encoding
  - Identify the position of each token
  - <u>Sinusoidal encoding</u>: calculate each dimension with a sinusoidal function, sin/cos function
  - Learn relative positions
    - PE(pos+k) can be expressed as a linear transformation of PE(pos)



2017 NeurIPs Attention Is All You Need

Transformer

- Positional encoding
  - Identify the position of each token
  - <u>Sinusoidal encoding</u>: calculate each dimension with a sinusoidal function, sin/cos function
  - Learn relative positions
  - Input: sum of token embeddings and positional encodings



2017 NeurIPs Attention Is All You Need

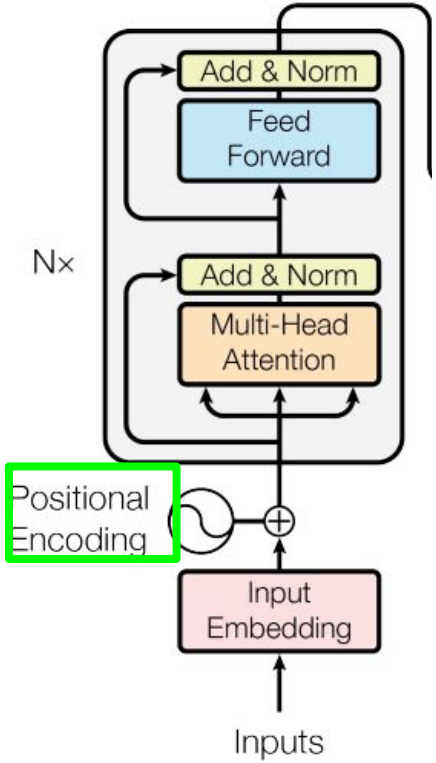
Transformer

- Positional encoding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

```
def _get_sinusoid_encoding_table(self, n_position, d_hid):
    ''' Sinusoid position encoding table '''
    # TODO: make it with torch instead of numpy

    def get_position_angle_vec(position):
        return [position / np.power(10000, 2 * (hid_j // 2) / d_hid) for hid_j in range(d_hid)]

    sinusoid_table = np.array([get_position_angle_vec(pos_i) for pos_i in range(n_position)]
    sinusoid_table[:, 0::2] = np.sin(sinusoid_table[:, 0::2])  # dim 2i
    sinusoid_table[:, 1::2] = np.cos(sinusoid_table[:, 1::2])  # dim 2i+1

    return torch.FloatTensor(sinusoid_table).unsqueeze(0)
```



2017 NeurIPs Attention Is All You Need

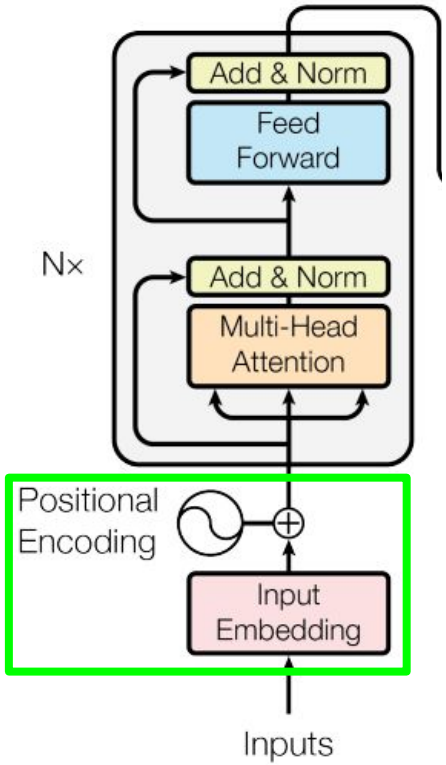
Transformer

- Scaled dot-produce self-attention
  - First: linear transformations of input word representations serves as the Q, K, V, respectively

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$$

  - Attention computation:

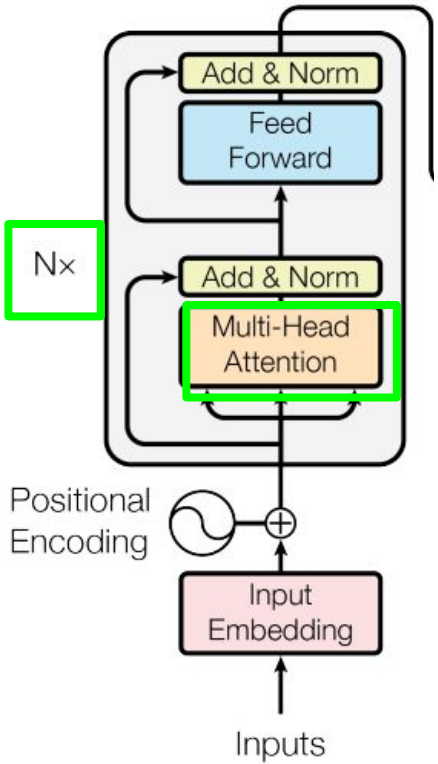
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



2017 NeurIPs Attention Is All You Need

Transformer

- Scaled dot-produce self-attention
  - Not only connect to input word representations
  - N blocks



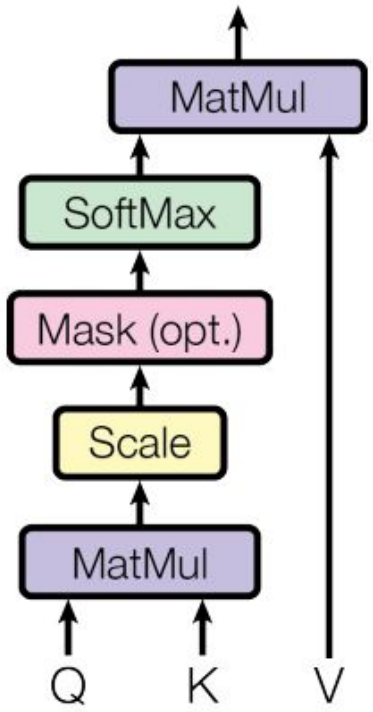
2017 NeurIPs Attention Is All You Need

Transformer

- Scaled dot-produce self-attention
  - Take the output from the previous layer which serves as the Q, K, V, respectively
    - **Standard Transformers: not shared!**

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$$

    - Each block learns different representation space
    - Earlier (syntatic patterns), middle (compositional structure), later (semantic feature)



2017 NeurIPs Attention Is All You Need

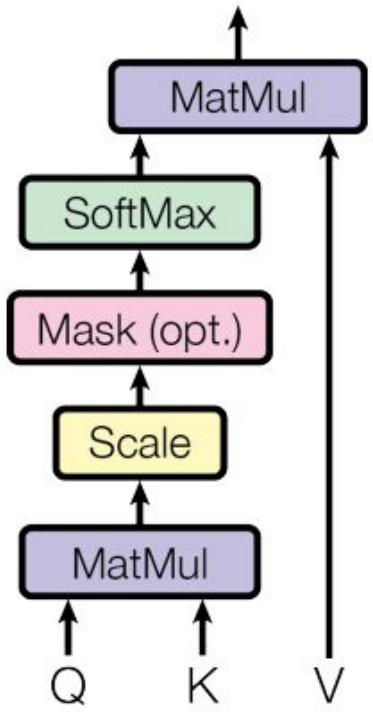
Transformer

- Scaled dot-produce self-attention
  - Take the output from the previous layer which serves as the Q, K, V, respectively
    - **Not shared! (exceptions, e.g., ALBERT)**

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$$

  - Attention computation:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



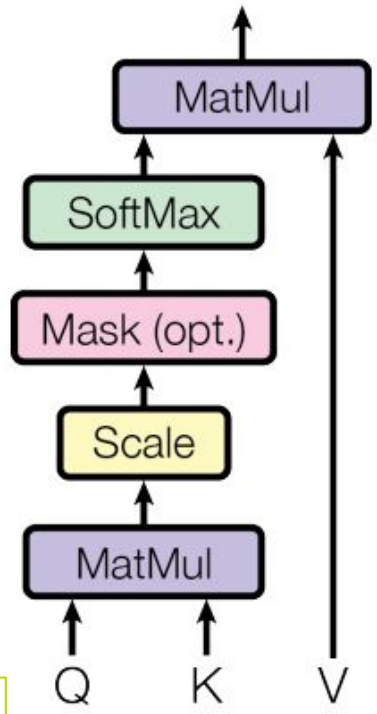
2017 NeurIPs Attention Is All You Need

Transformer

- Scaled dot-produce self-attention
  - Take the output from the previous layer which serves as the Q, K, V, respectively
  - Attention computation:

```python
def forward(self, q, k, v, mask=None):

    attn = torch.matmul(q / self.temperature, k.transpose(2, 3))

    if mask is not None:
        attn = attn.masked_fill(mask == 0, -1e9)

    attn = self.dropout(F.softmax(attn, dim=-1))
    output = torch.matmul(attn, v)

    return output, attn
```

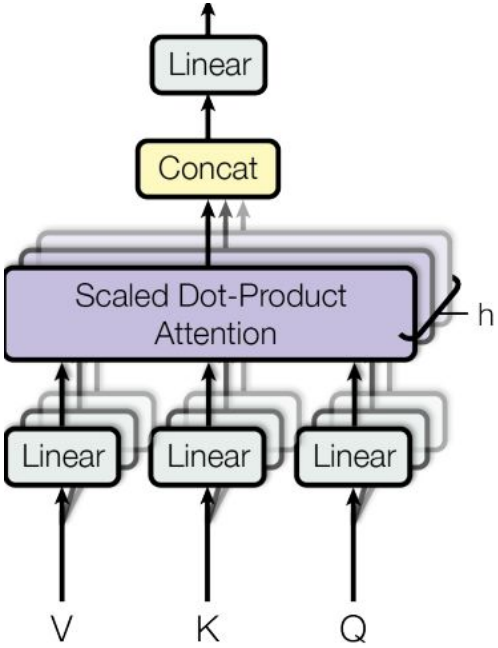
```python
self.attention = ScaledDotProductAttention(temperature=d_k ** 0.5)
```



2017 NeurIPs Attention Is All You Need

Transformer

- Multi-head attention:
  - Multiple attention heads (h) each attend to different parts individually
    - Different parts of a sentence may provide different types of contextual signals for disambiguation

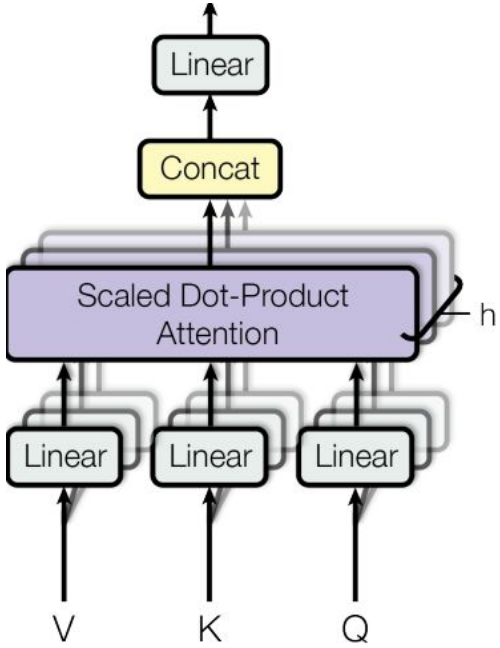


2017 NeurIPs Attention Is All You Need

Transformer

- Multi-head attention:
  - Multiple attention heads (h) each attend to different parts individually
    - Different parts of a sentence may provide different types of contextual signals for disambiguation

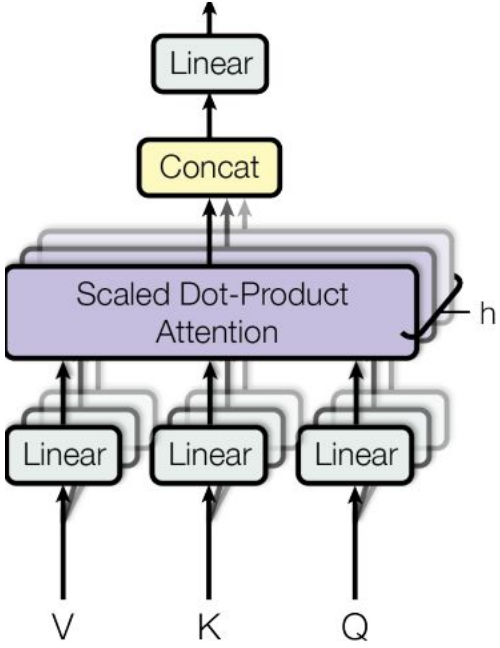
The animal didn't cross the street because it was too tired.



2017 NeurIPs Attention Is All You Need

Transformer

- Multi-head attention:
  - Catch different relational patterns simultaneously
  - different learned representation subspace
    - Head 1: long dependency, distant noun
    - Head 2: semantic plausibility
    - Head 3: syntactic structure: nearby noun phrase



The animal didn't cross the street because it was too tired.

2017 NeurIPs Attention Is All You Need

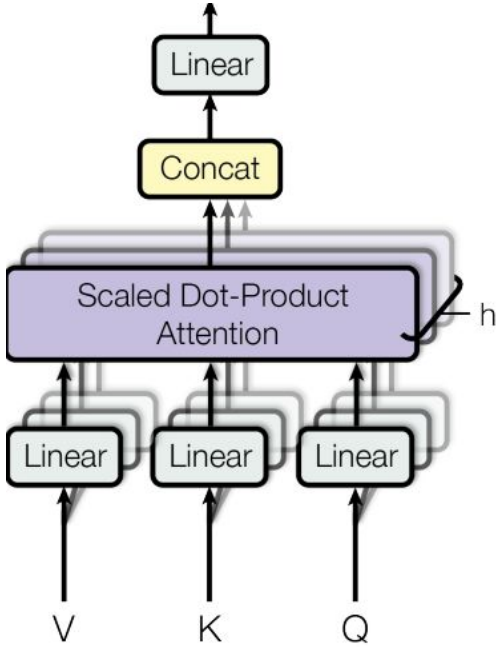
Transformer

- Multi-head attention:
  - Multiple attention heads (h) each attend to different parts individually: h maps
  - Run individual attention head

$$\mathrm{head_i} = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

  - Stack all outputs together

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head_1}, ..., \mathrm{head_h})W^O$$

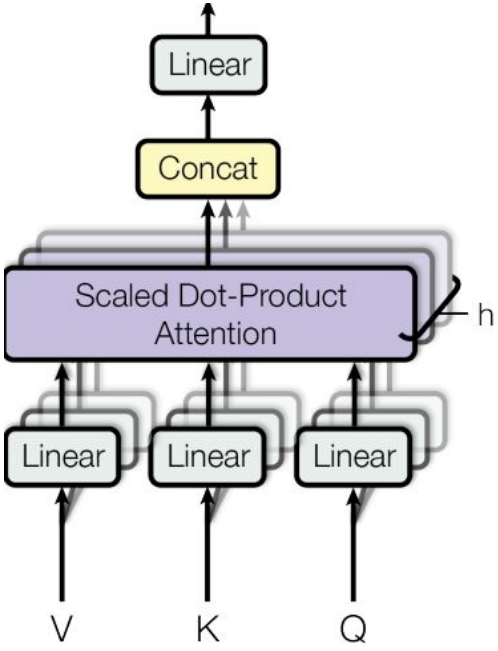


2017 NeurIPs Attention Is All You Need

Transformer

- Multi-head attention:

```
# Pass through the pre-attention projection: b x lq x (n*dv)
# Separate different heads: b x lq x n x dv
q = self.w_qs(q).view(sz_b, len_q, n_head, d_k)
k = self.w_ks(k).view(sz_b, len_k, n_head, d_k)
v = self.w_vs(v).view(sz_b, len_v, n_head, d_v)

# Transpose for attention dot product: b x n x lq x dv
q, k, v = q.transpose(1, 2), k.transpose(1, 2), v.transpose(1, 2)

if mask is not None:
    mask = mask.unsqueeze(1)   # For head axis broadcasting.

q, attn = self.attention(q, k, v, mask=mask)

# Transpose to move the head dimension back: b x lq x n x dv
# Combine the last two dimensions to concatenate all the heads together: b x lq x (n*dv)
q = q.transpose(1, 2).contiguous().view(sz_b, len_q, -1)
```

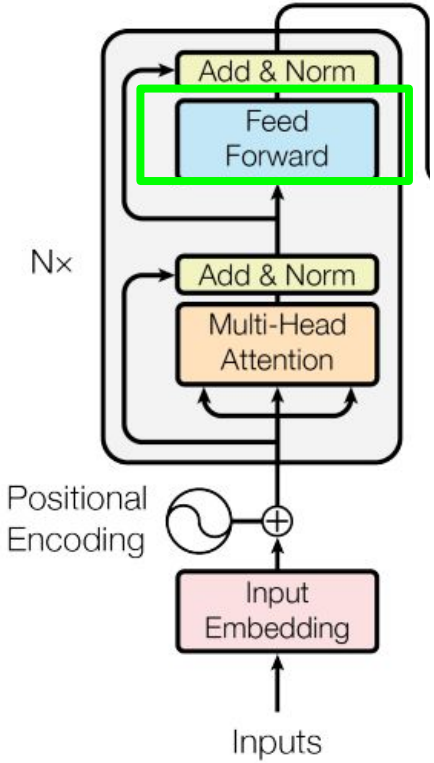


2017 NeurIPs Attention Is All You Need

Transformer

- Feed forward networks (FFN)
  - Extract features from the attended output
  - Non-linearity: ReLU function

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



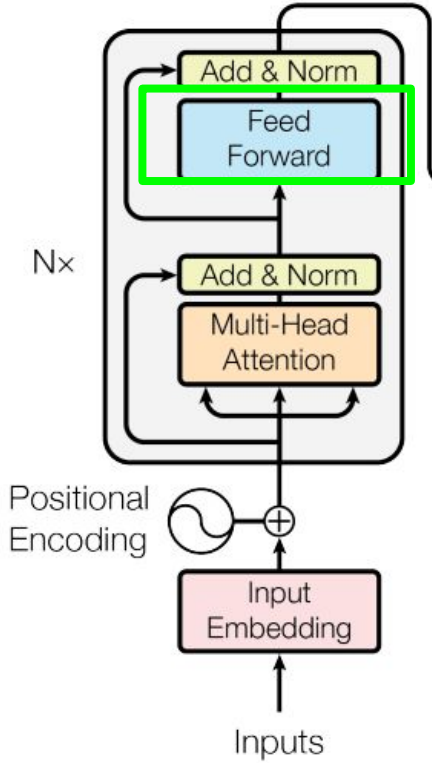
2017 NeurIPs Attention Is All You Need

Transformer

- Feed forward networks (FFN)

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

```python
def __init__(self, d_in, d_hid, dropout=0.1):
    super().__init__()
    self.w_1 = nn.Linear(d_in, d_hid) # position-wise
    self.w_2 = nn.Linear(d_hid, d_in) # position-wise
    self.layer_norm = nn.LayerNorm(d_in, eps=1e-6)
    self.dropout = nn.Dropout(dropout)

def forward(self, x):

    residual = x

    x = self.w_2(F.relu(self.w_1(x)))
    x = self.dropout(x)
    x += residual

    x = self.layer_norm(x)

    return x
```

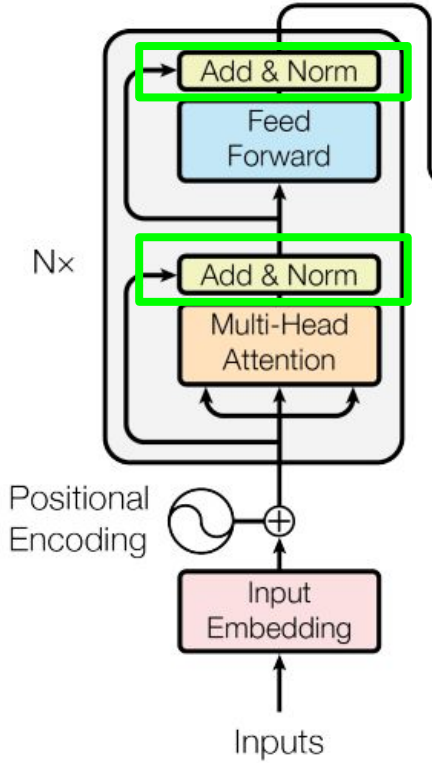


2017 NeurIPs Attention Is All You Need

Transformer

- Residual and layer norm
  - Applied around each of the two sub-layers
    - Multi-head attention
    - Feed forward network
  - Output of each <u>sub-layer</u>

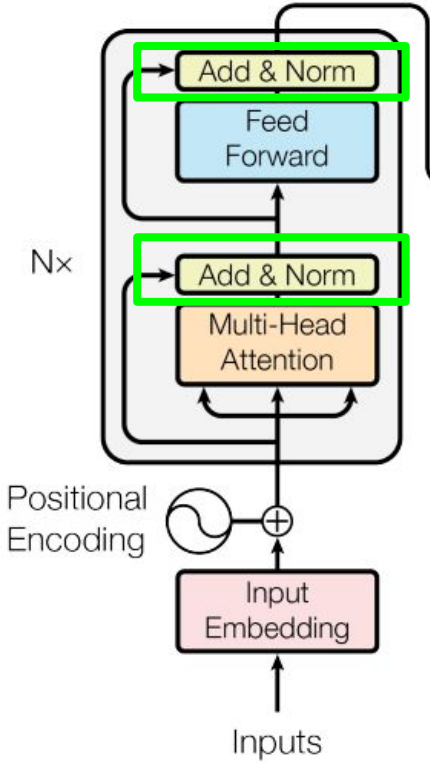
$$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$$



2017 NeurIPs Attention Is All You Need

Transformer

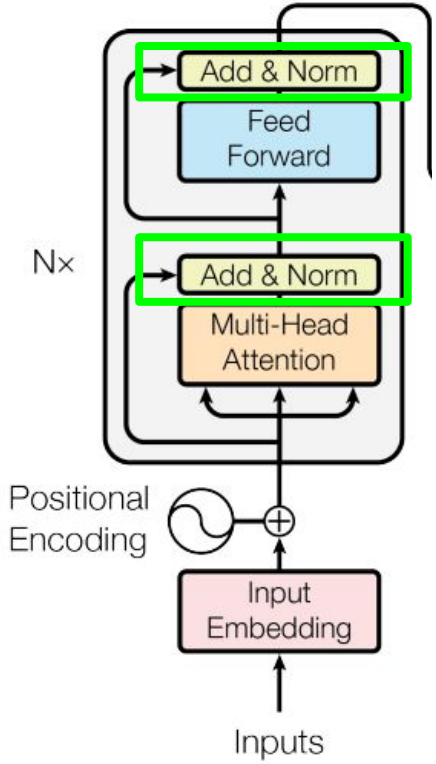
- Residual and layer norm
  - Applied around each of the two sub-layers
  - Motivation for residual:
    - Prevent vanishing gradient
    - Only learns how to update inputs, rather than completely transform inputs
    - Preserve information from earlier layers
    - Enable deep architectures (CNN)



2017 NeurIPs Attention Is All You Need

Transformer

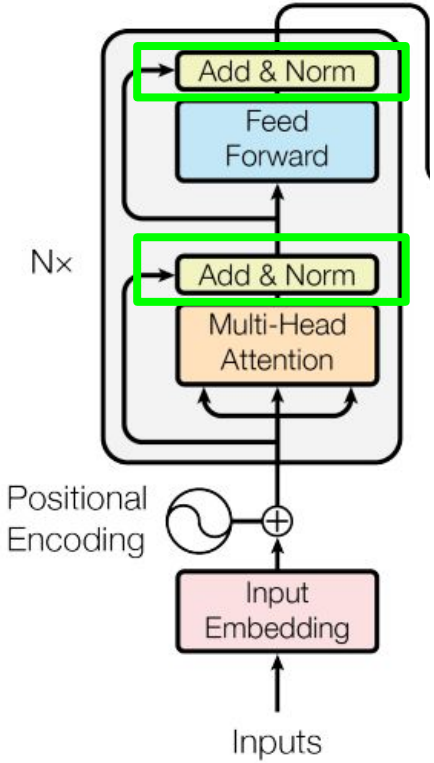
- Residual and layer norm
  - Motivation for Layernorm:
    - Ensure outputs in a layer in a consistent range
    - Preventing too much variance in scale of outputs



2017 NeurIPs Attention Is All You Need

Transformer

- Residual and layer norm
  - Layernorm: $\gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$



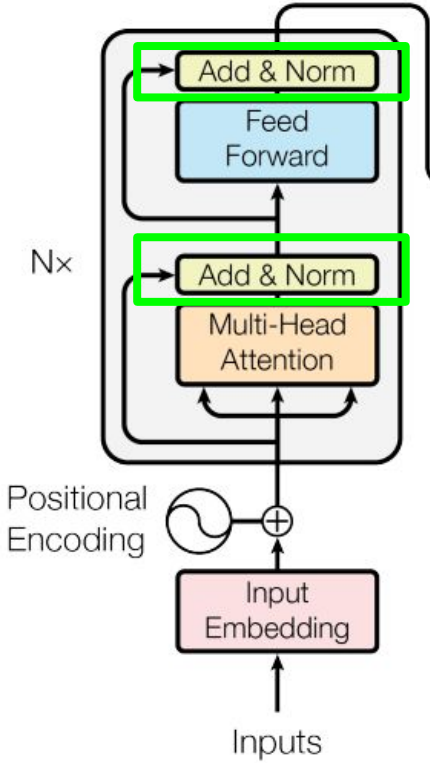
2017 NeurIPs Attention Is All You Need

Transformer

- Residual and layer norm
  - Layernorm: $\gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$

```
self.layer_norm = nn.LayerNorm(d_model, eps=1e-6)
```

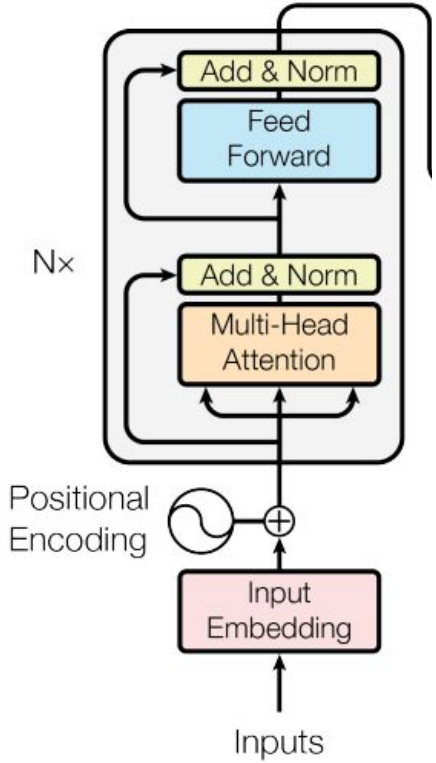
```
q += residual


q = self.layer_norm(q)
```



2017 NeurIPs Attention Is All You Need

Transformer

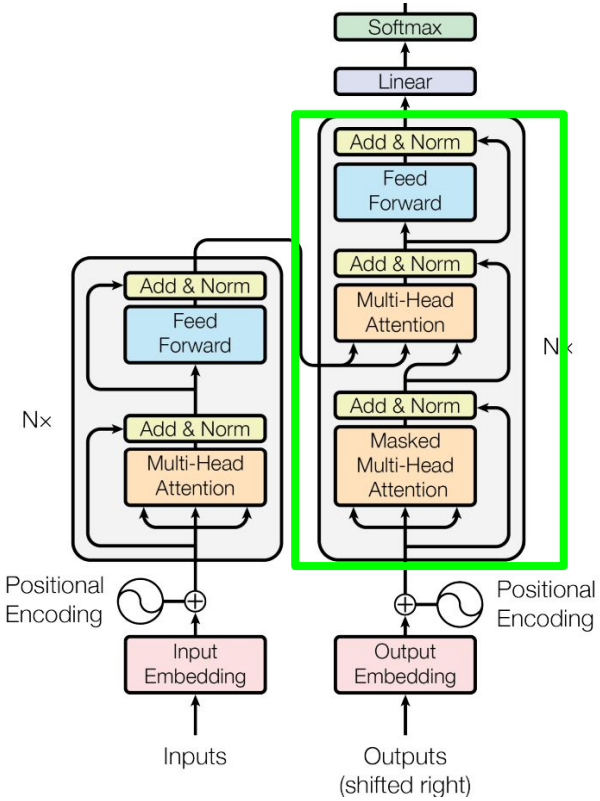
- Encoder: stack of N=6 identical layers
  - Two sub-layers: residual & Layernorm connection
    - Multi-head attention
    - FFN



2017 NeurIPs Attention Is All You Need

Transformer

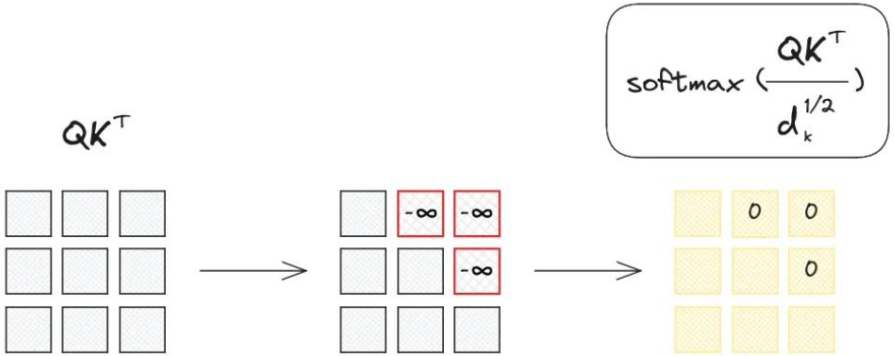
- Decoder: stack of N=6 identical layers
  - Three sub-layers
  - Masked multi-head attention
    - Output embeddings are offset by one position

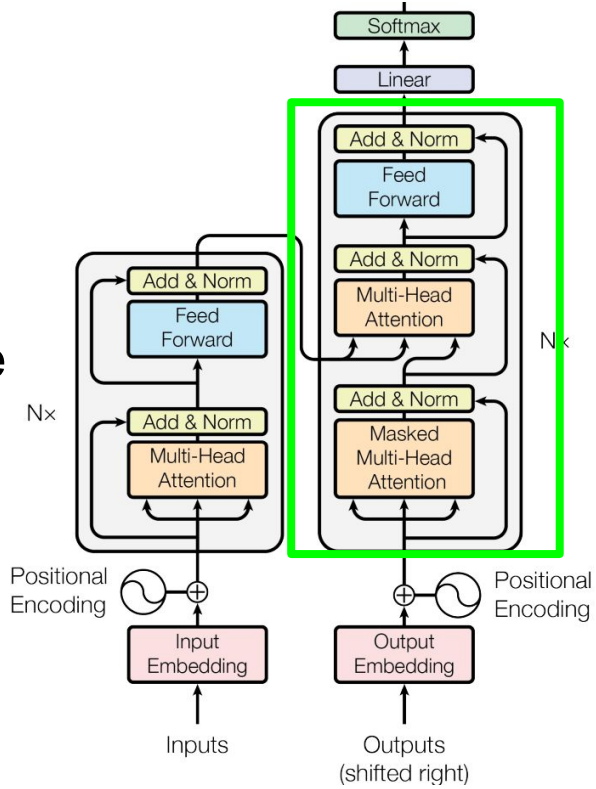


2017 NeurIPs Attention Is All You Need

Transformer

- Decoder: stack of N=6 identical layers
  - Three sub-layers
  - Masked multi-head attention
    - All values above the diagonal are replaced with a negative infinity value

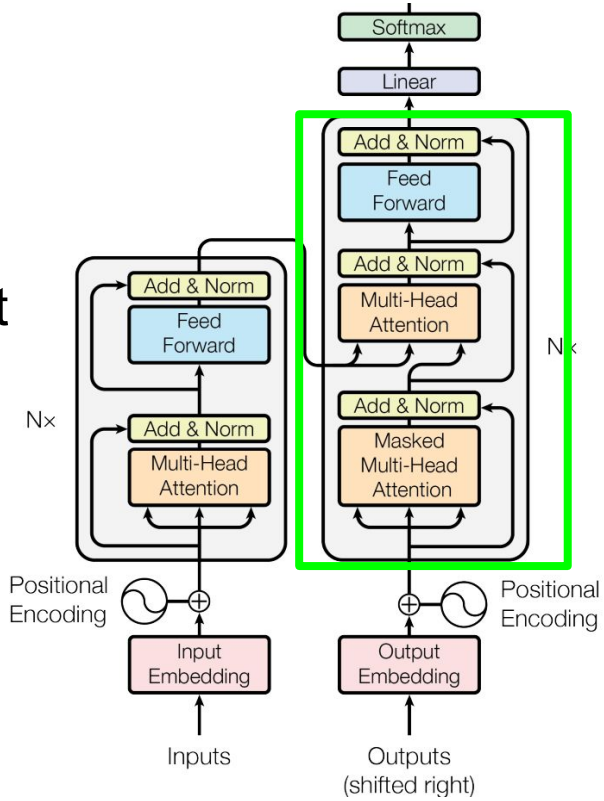




image

Transformer

- Decoder: stack of N=6 identical layers
  - Three sub-layers
  - Masked multi-head attention
    - Ensure that the predictions for current position can depend only on previous tokens
      - Inputs and previously generated tokens

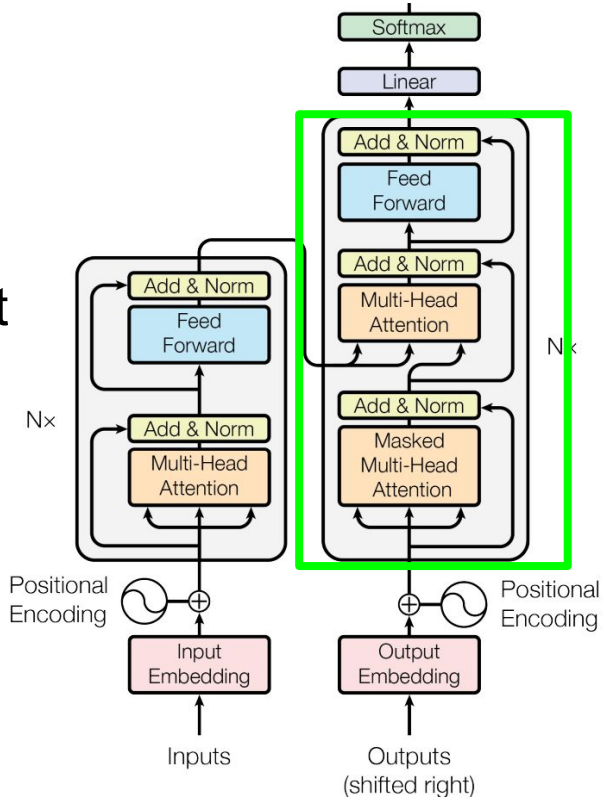


2017 NeurIPs Attention Is All You Need

Transformer

- Decoder: stack of N=6 identical layers
  - Three sub-layers
  - Masked multi-head attention
    - Ensure that the predictions for current position can depend only on previous tokens
      - Inputs and previously generated tokens

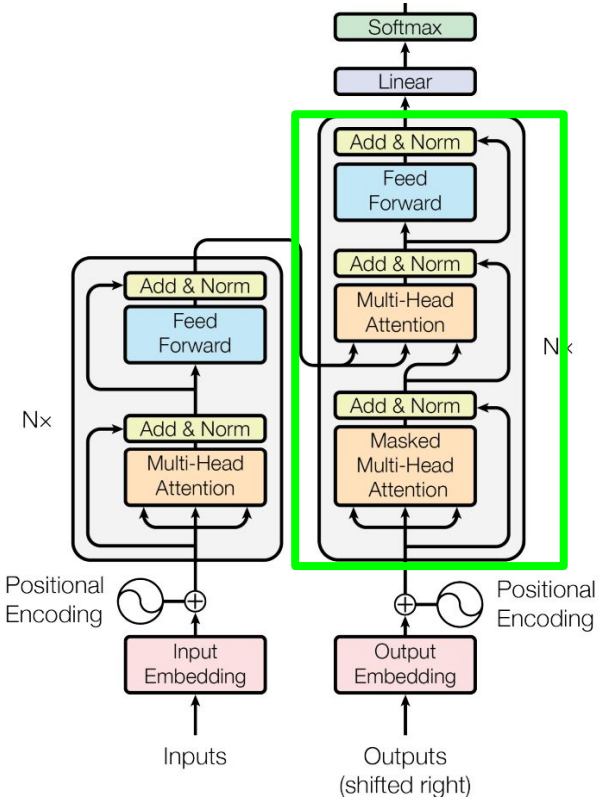
Why we need it?



2017 NeurIPs Attention Is All You Need

Transformer

- Decoder: stack of N=6 identical layers
  - Three sub-layers
  - Masked multi-head attention
    - Training in parallel
    - Next token prediction is based on the **GT** previous token



2017 NeurIPs Attention Is All You Need

Improvements over Transformer

- Positional encoding
  - Transformer: fixed function encoding, weak relative position encoding
  - Learned positional vector?
    - Length extrapolation: hard to generalize to longer sequences

2021 NeurIPs RoFormer: Enhanced Transformer with Rotary Position Embedding

Improvements over Transformer

- Positional encoding
  - Transformer: fixed function encoding, weak relative position encoding
  - RoPE proposed in RoFormer (used by LLaMA, Mistral)
    - Injects position information by rotating the query and key vectors in attention space

2021 NeurIPs RoFormer: Enhanced Transformer with Rotary Position Embedding

Improvements over Transformer

- Multi query attention
  - Goal:
    - Reduce memory and computational cost

Improvements over Transformer

- Multi query attention (MQA)
  - Standard multi-**<u>head</u>** attention (MHA)
    - Each head has its own query, key, and value projections

$$Q_i = XW_Q^{(i)}, \quad K_i = XW_K^{(i)}, \quad V_i = XW_V^{(i)}$$

  - MQA
    - Each head still has its own query
    - All heads share the same key and value

$$Q_i = XW_Q^{(i)}$$

$$K = XW_K$$

$$V = XW_V$$

Multi query attention

Improvements over Transformer

- Grouped query attention: used by LLaMA2
  - Goal:
    - Reduce memory and computational cost
    - Maintain the performance of standard multi-head attention

Improvements over Transformer

- Grouped query attention: used by LLaMA2
  - Middle ground between MHA and MQA
  - Heads first divided into groups
  - Each group shares one key and value

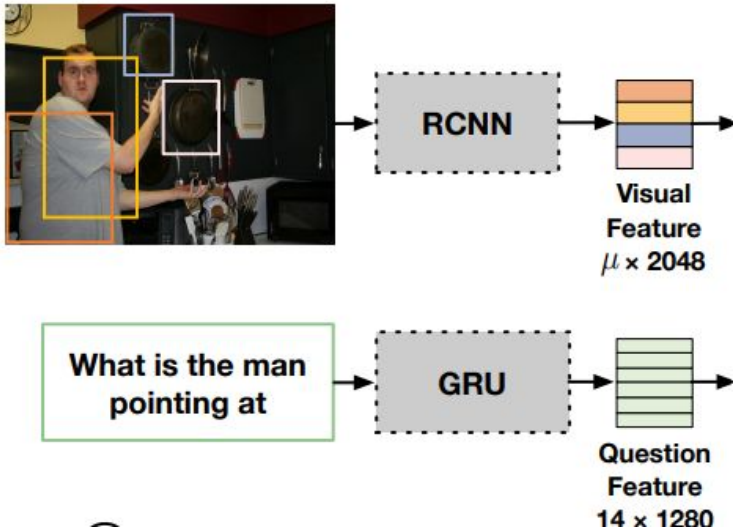
2023 EMNLP GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

Improvements over Transformer

- Grouped query attention: used by LLaMA2
  - Middle ground between MHA and MQA
  - Heads first divided into groups
  - Each group shares one key and value
  - In other words
    - Query projection: per head
    - Key/value projection: per group

Application of Transformer

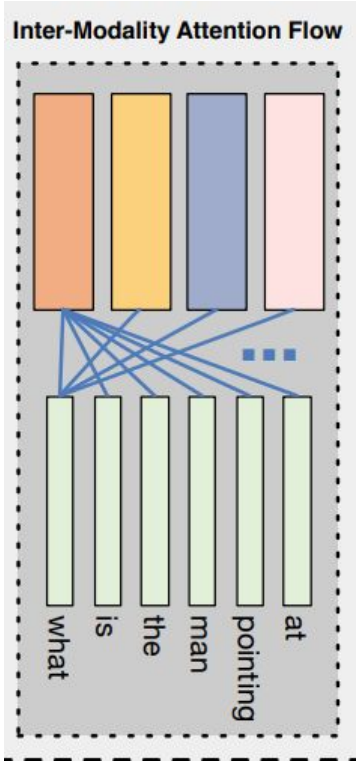
- For Image QA
  - Two input sequences:
    - Image features
    - Question encoding representations
      - Parallel computation



2019 CVPR Dynamic Fusion with Intra- and Inter-modality Attention Flow for Visual Question Answering

Application of Transformer

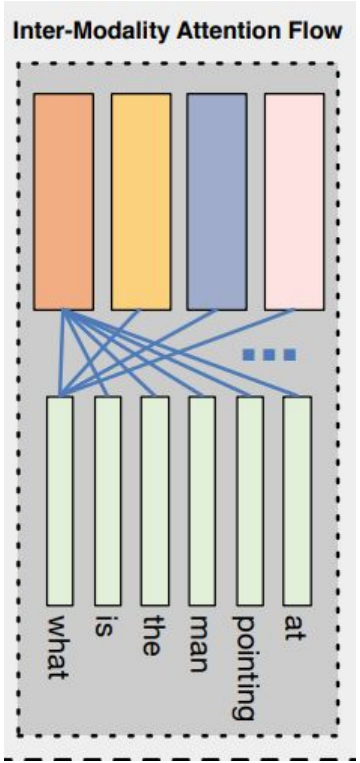
- For Image QA
  - Inter-modality attention
  - Goal:
    - Relevant information flow from the other modality
      - E.g,. given the question word “man”, highlight the relevant image region



2019 CVPR Dynamic Fusion with Intra- and Inter-modality Attention Flow for Visual Question Answering

Application of Transformer

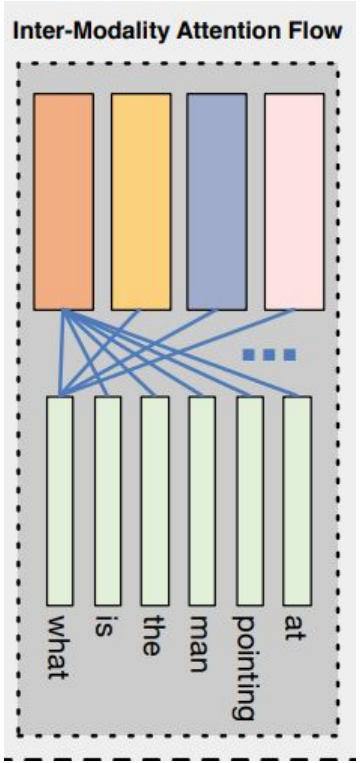
- For Image QA
  - Inter-modality attention
  - Question related image parts:
    - Query: question vectors
    - Key: image vectors
    - Value: image vector



2019 CVPR Dynamic Fusion with Intra- and Inter-modality Attention Flow for Visual Question Answering

Application of Transformer

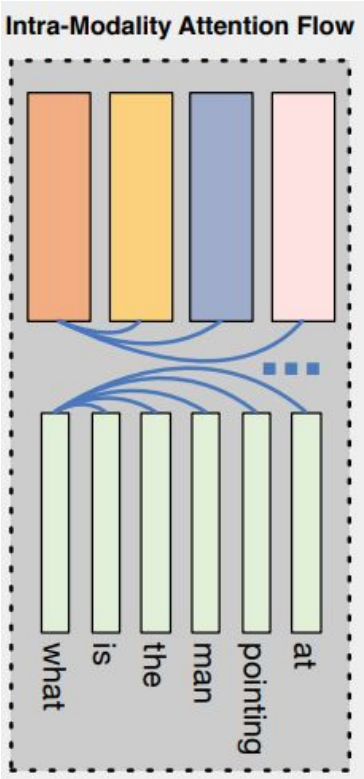
- For Image QA
  - Inter-modality attention
  - Question related image parts
  - Augment the question word with representation from relevant image region
    - Multimodal interaction: bridge the modality gap



2019 CVPR Dynamic Fusion with Intra- and Inter-modality Attention Flow for Visual Question Answering

Application of Transformer

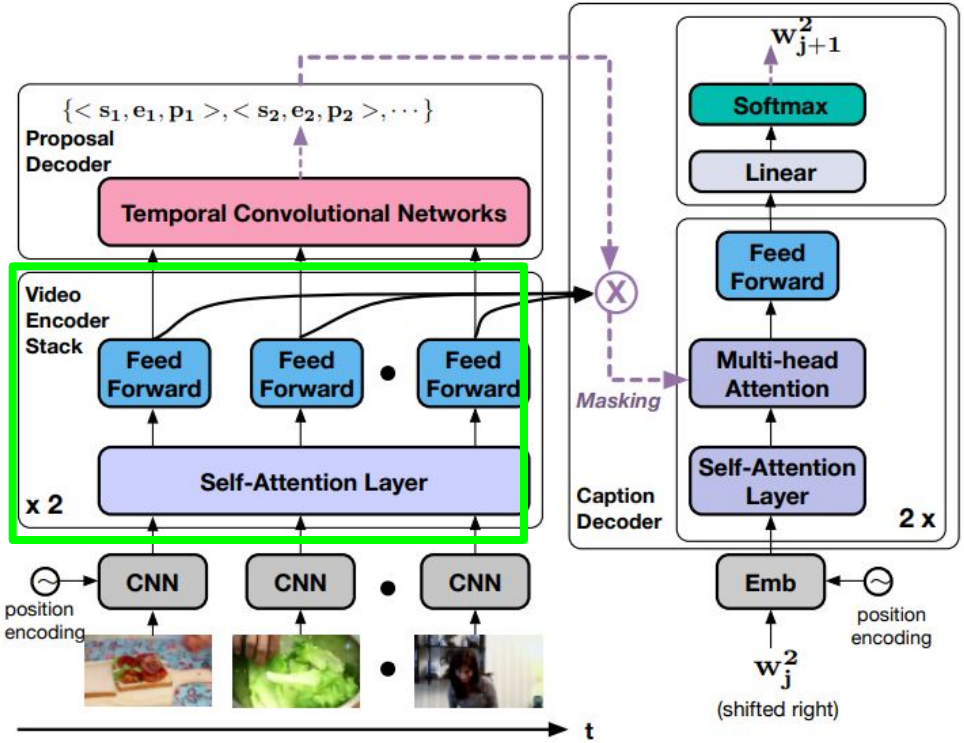
- For Image QA
  - Intra-modality attention
  - Self-attention
  - Highlight most salient parts:
    - E.g., ignore stopwords in questions



2019 CVPR Dynamic Fusion with Intra- and Inter-modality Attention Flow for Visual Question Answering

Application of Transformer

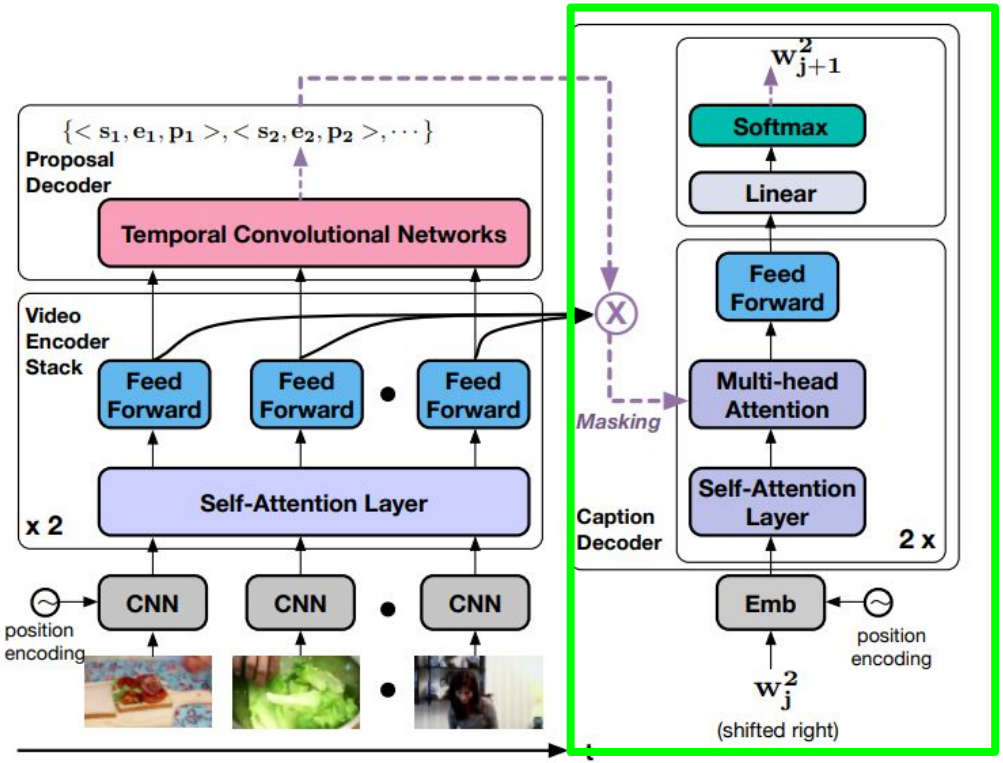
- Video captioning
  - Self-attention over input video frames



2018 CVPR End-to-End Dense Video Captioning with Masked Transformer

Application of Transformer

- Video captioning
  - Self-attention over input video frames
  - Masked attention for caption generation



2018 CVPR End-to-End Dense Video Captioning with Masked Transformer

ANY QUESTION